Data Structures

(Multi-Dimensional Array and Linked List)

Naeem Ahmed
(Slides adapted from Mushtari Sadia)

Introduction

Faculty Name: Naeem Ahmed (NMA)
Room: UB80601

Consultation Hours:

Sunday: 9:30 AM - 11:00 AM

Monday: 11:00 AM-12:30 PM

Wednesday: 11:00 AM-2:00 PM

Linked List

Limitations of Array

● **Fixed capacity :** Once created, an array cannot be resized. The only way to "resize" is to create a larger new array, copy the elements from the original array into the new one, and then change the reference to the new one.

Limitations of Array

● **Fixed capacity :** Once created, an array cannot be resized. The only way to "resize" is to create a larger new array, copy the elements from the original array into the new one, and then change the reference to the new one.

● **Shifting elements in insert :** Since we do not allow gaps in the array, inserting a new element requires that we shift all the subsequent elements right to create a hole where the new element is placed. In the worst case, we "disturb" every slot in the array when we insert it at the beginning of the array!

Limitations of Array

● **Fixed capacity :** Once created, an array cannot be resized. The only way to "resize" is to create a larger new array, copy the elements from the original array into the new one, and then change the reference to the new one.

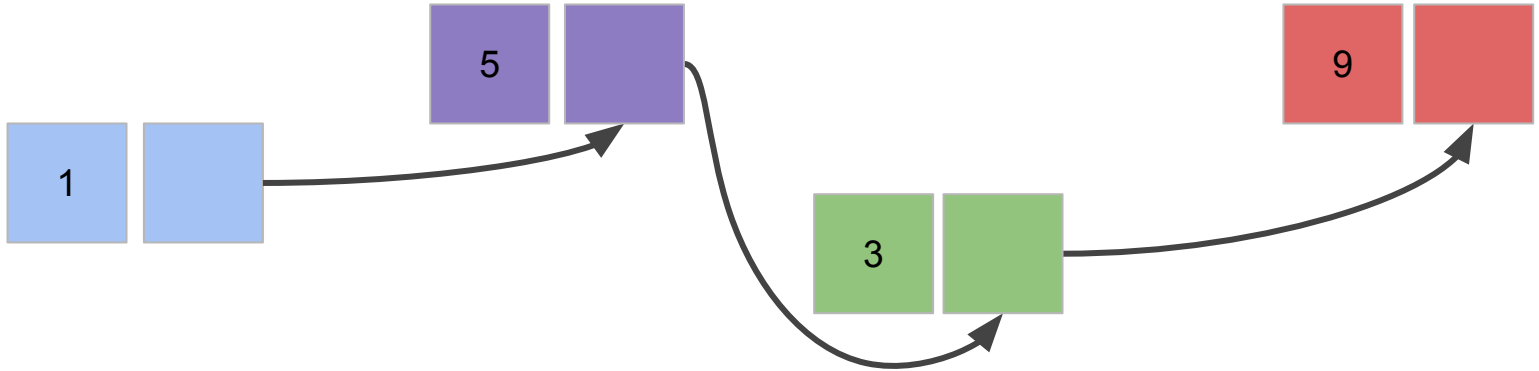
● **Shifting elements in insert :** Since we do not allow gaps in the array, inserting a new element requires that we shift all the subsequent elements right to create a hole where the new element is placed. In the worst case, we "disturb" every slot in the array when we insert it at the beginning of the array!

● **Shifting elements in removal** : Removing may also require shifting to plug the hole left by the removed element. If the ordering of the elements does not matter, we can avoid shifting by replacing the removed element with the element in the last slot (and then treating the last slot as empty).

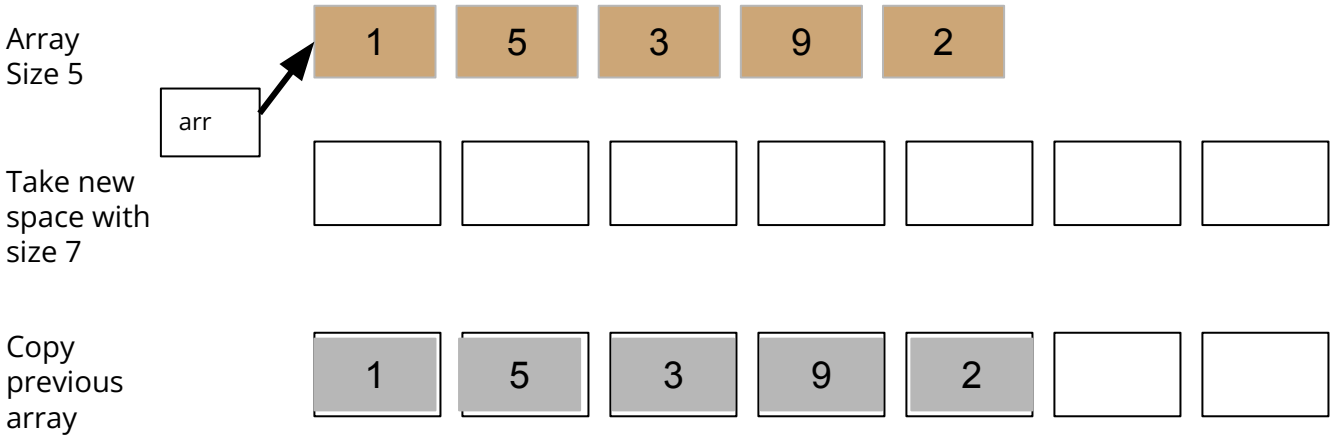
Array vs. Linked List

Array
in memory

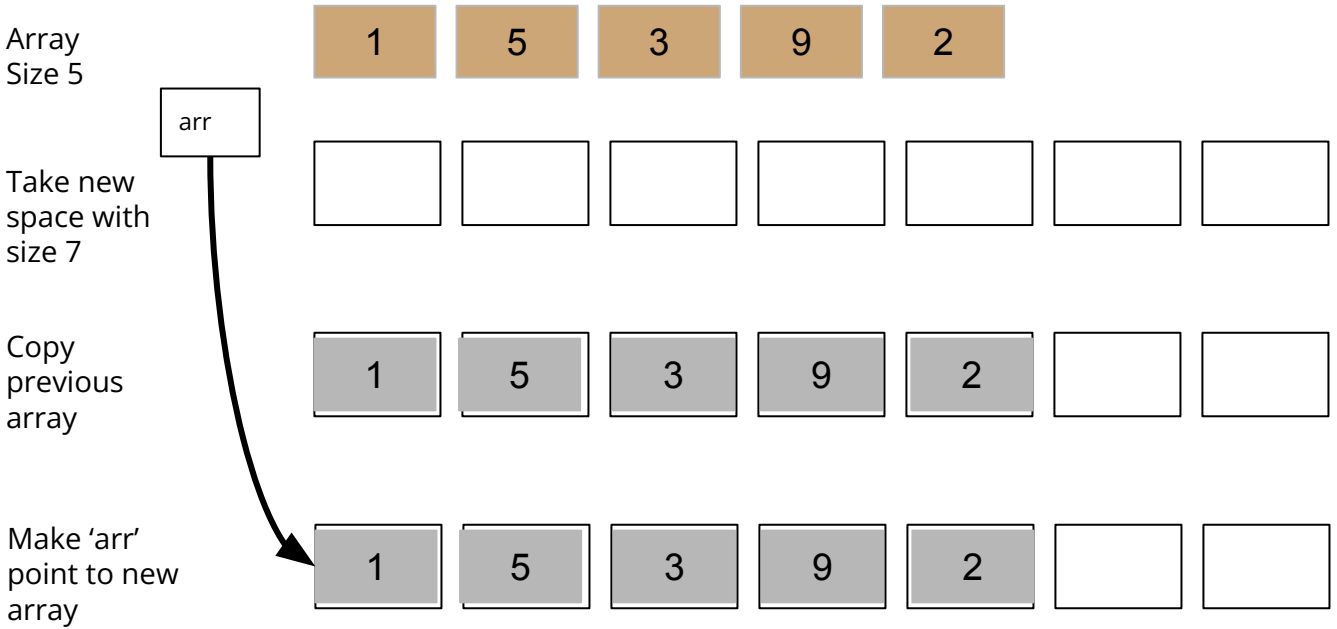
Linked list
in memory



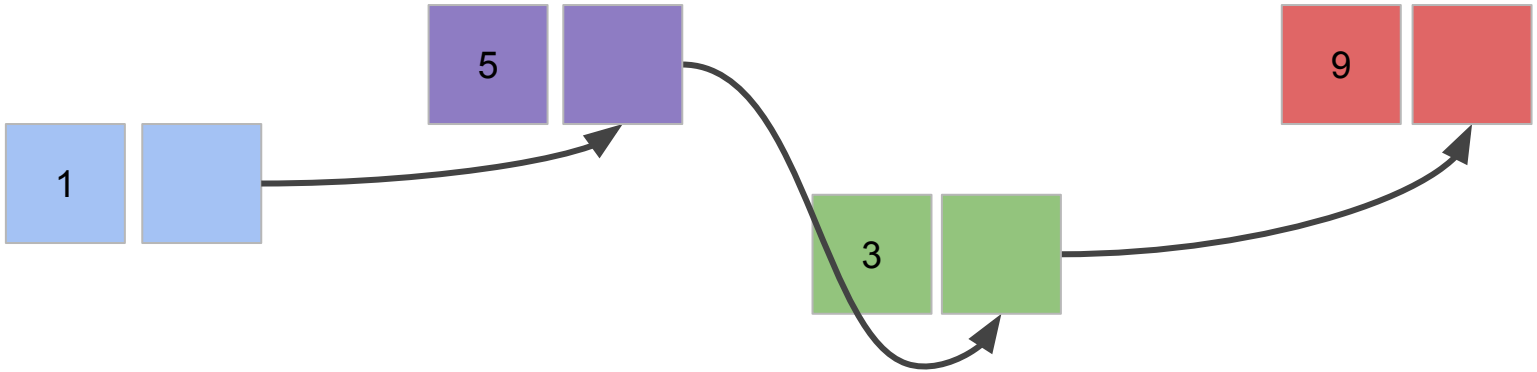
Resizing - Array



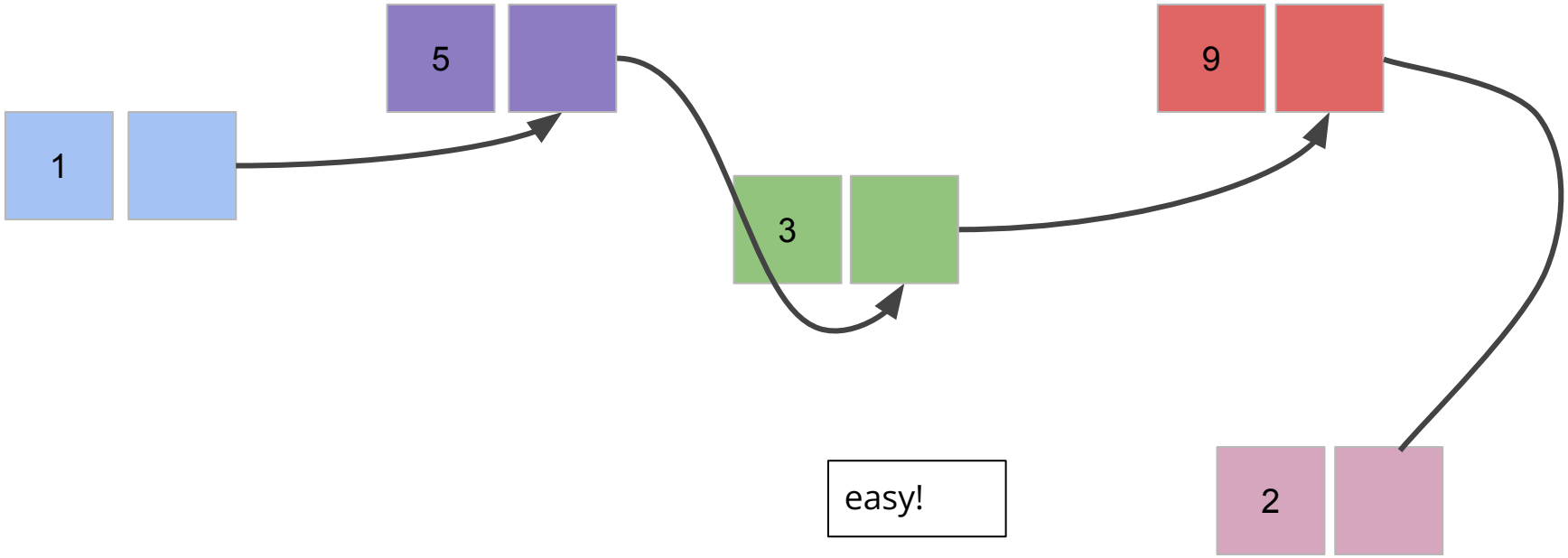
Resizing - Array



Resizing - Linked List

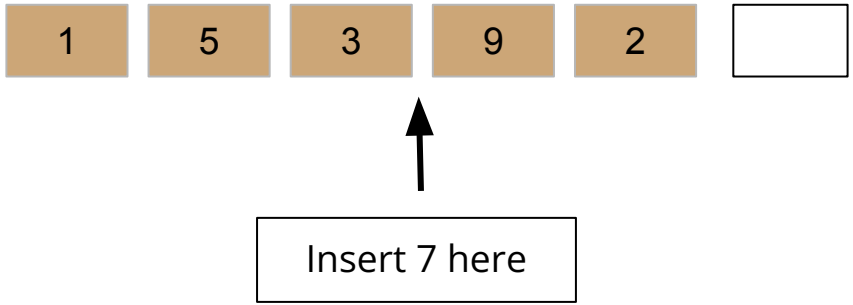


Resizing - Linked List

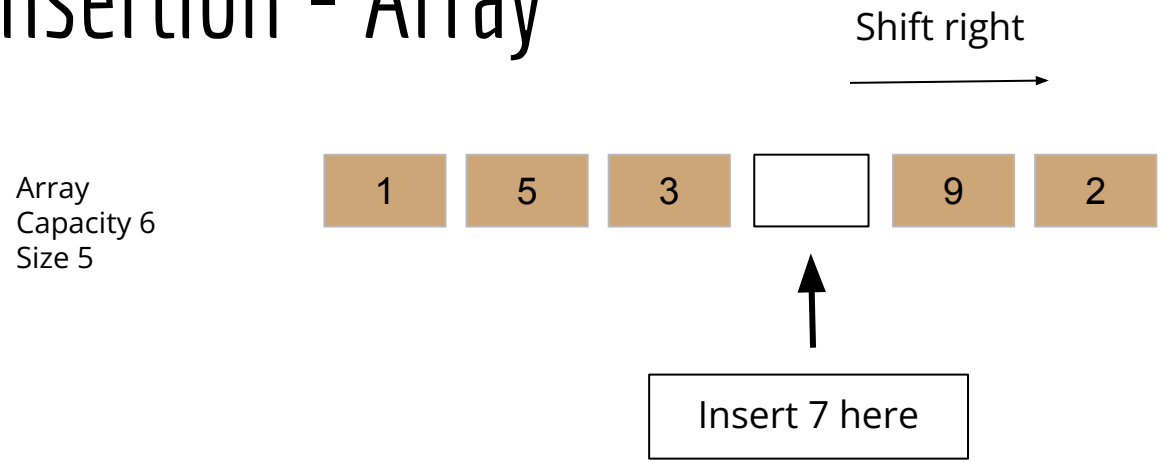


Insertion - Array

Array
Capacity 6
Size 5

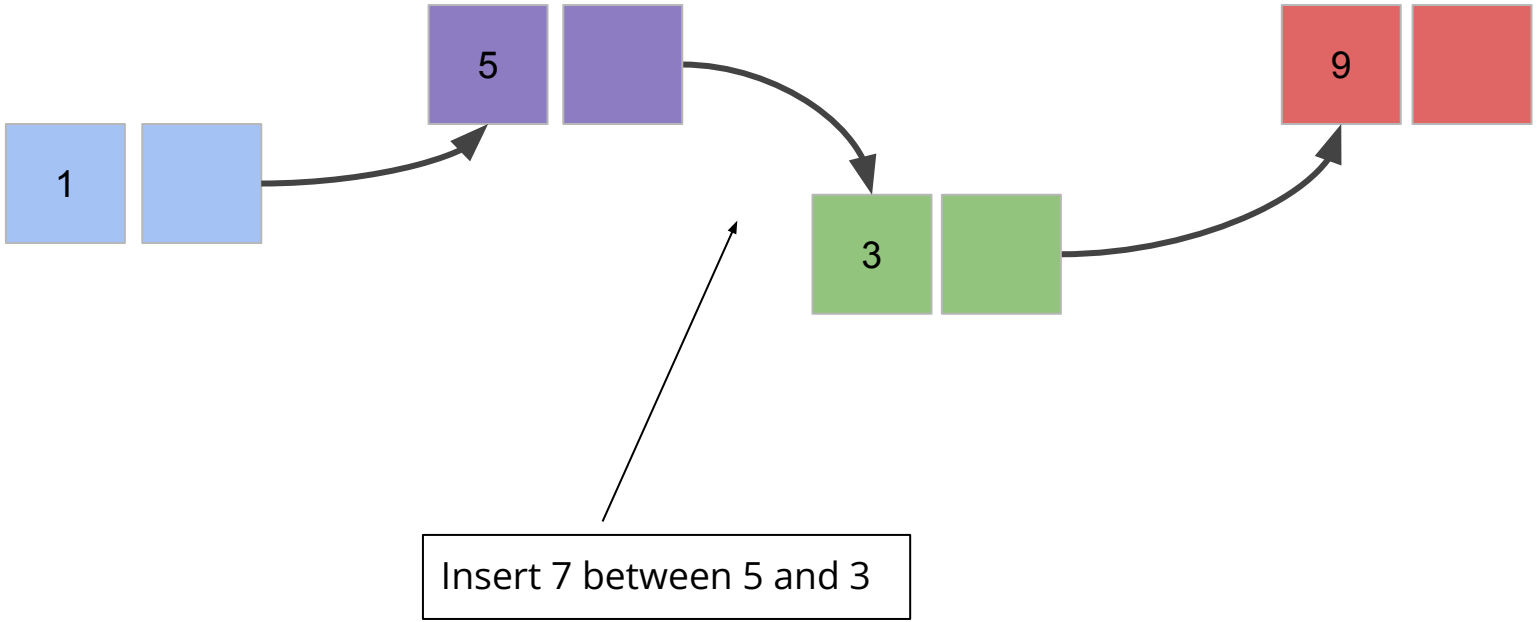


Insertion - Array

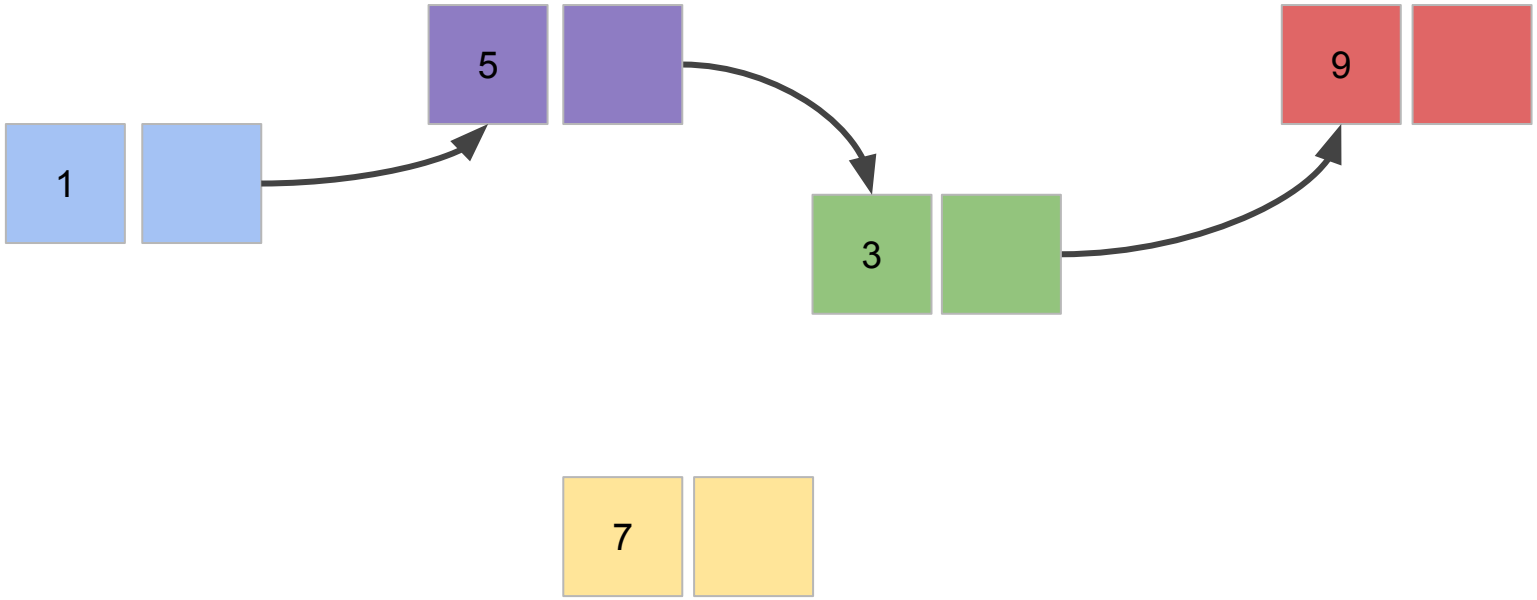


array[3] = 7

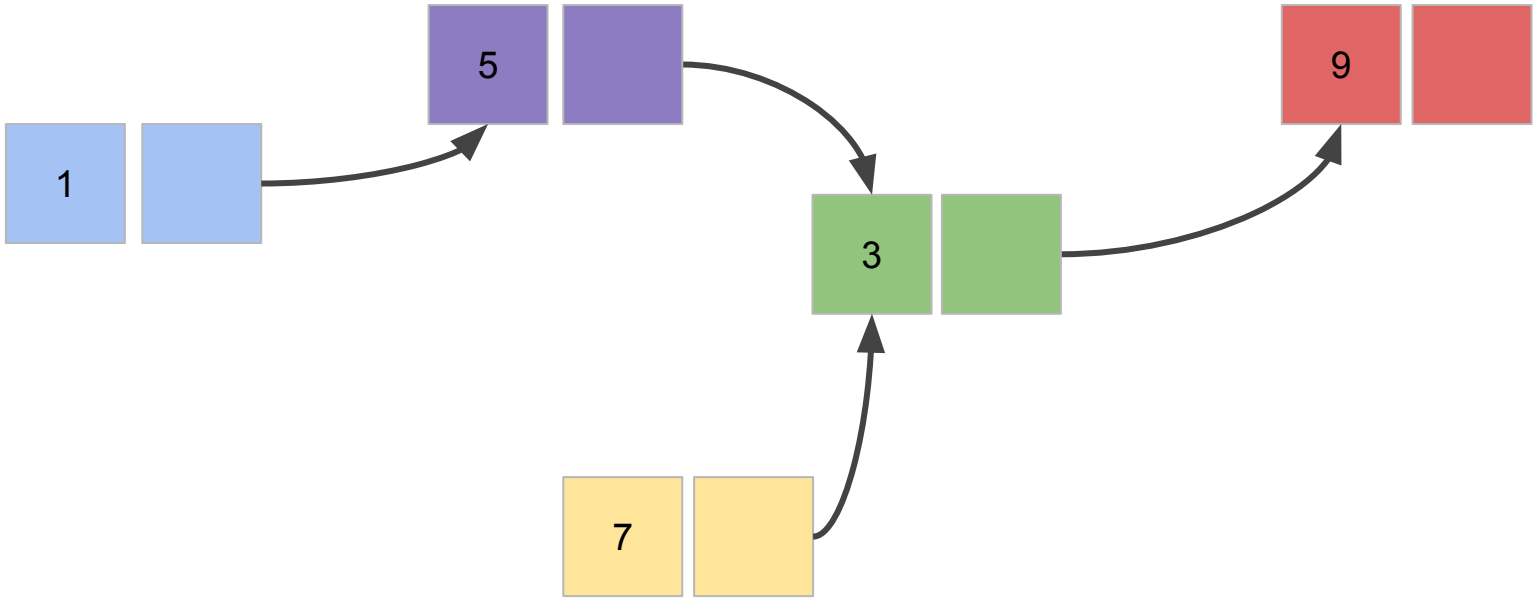
Insertion - Linked List



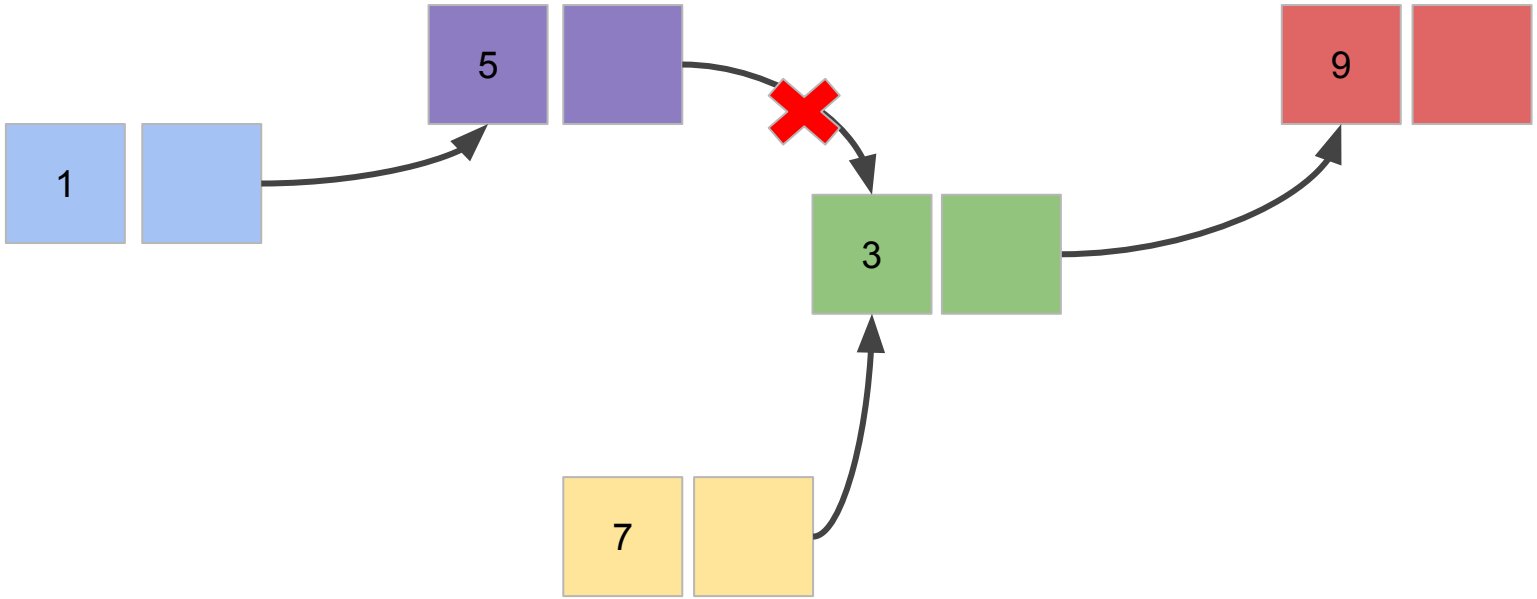
Insertion - Linked List



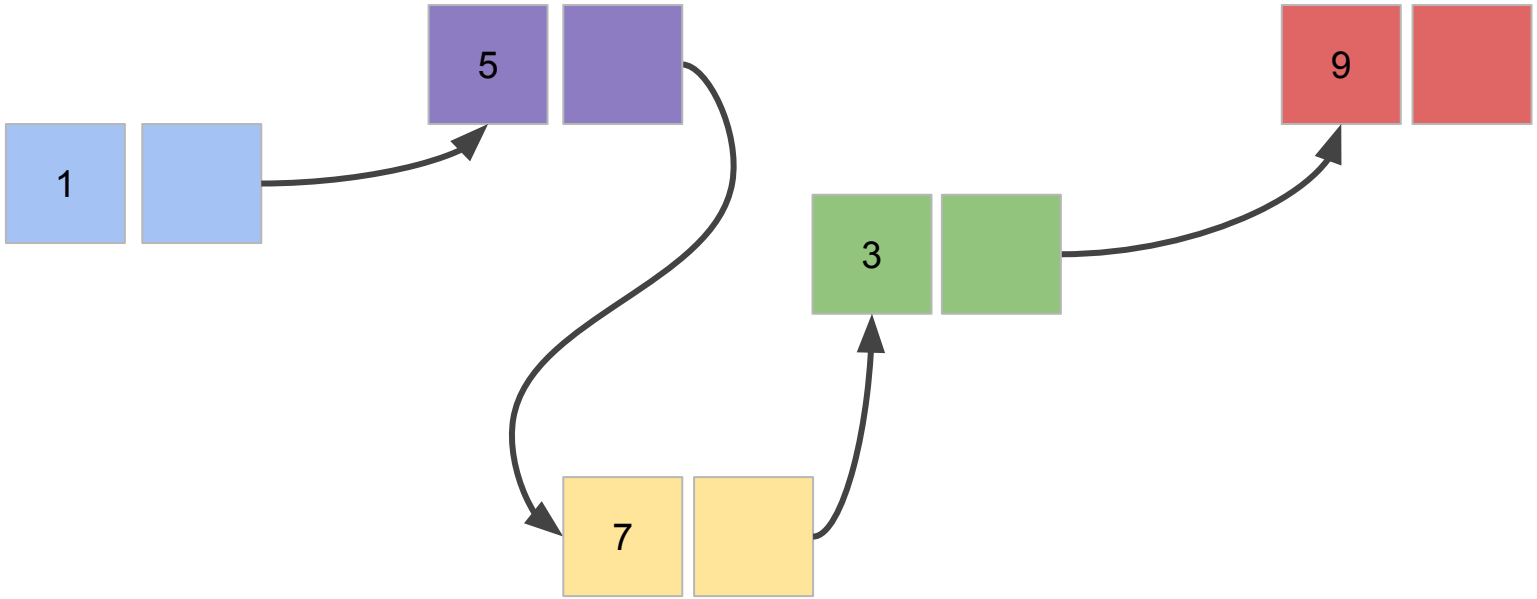
Insertion - Linked List



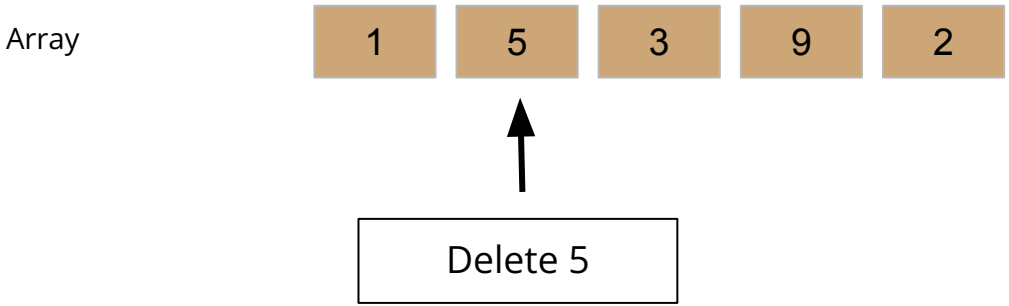
Insertion - Linked List



Insertion - Linked List



Removal - Array

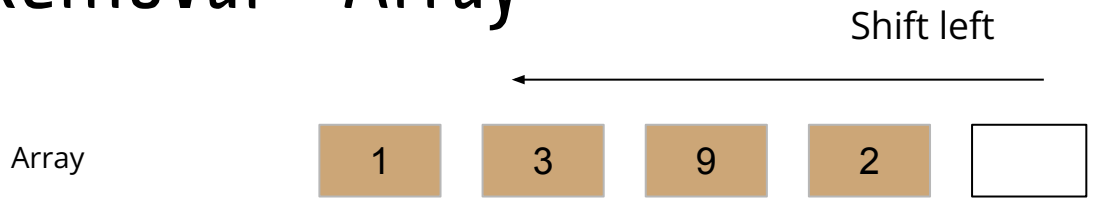


Removal - Array

Array

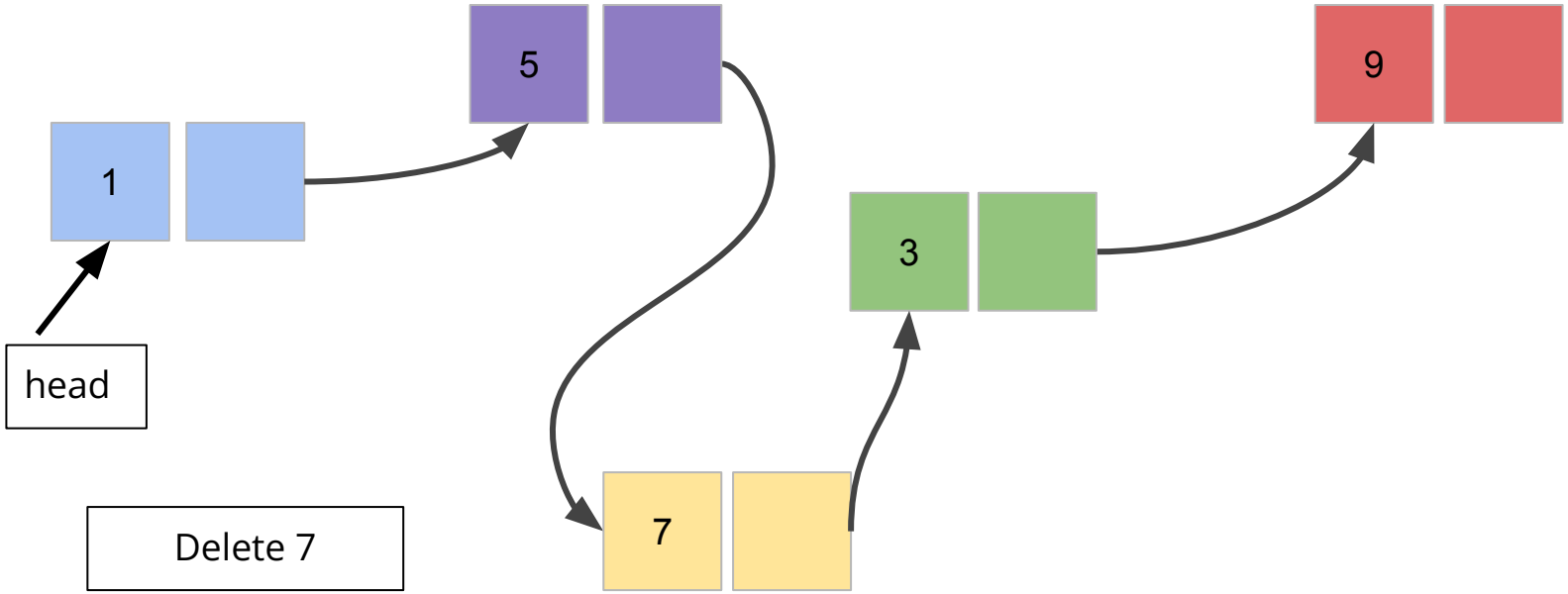
array[1] = None

Removal - Array

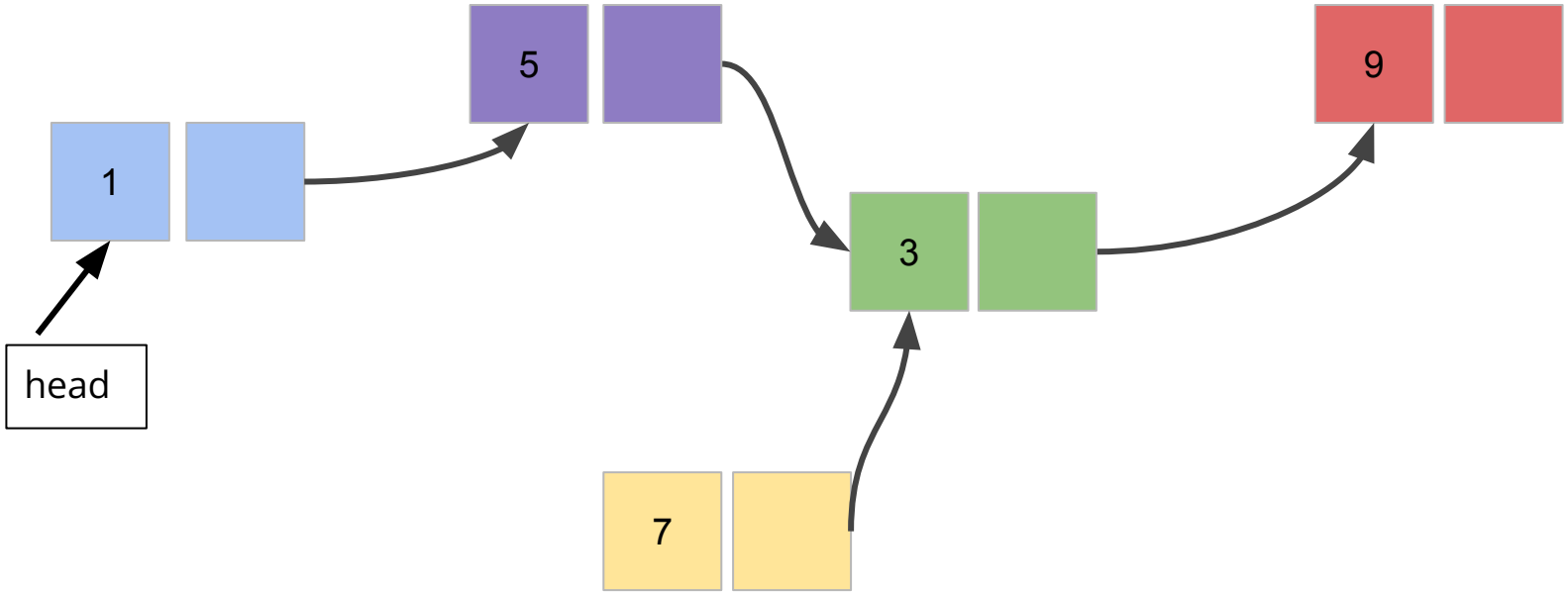


array[1] = None

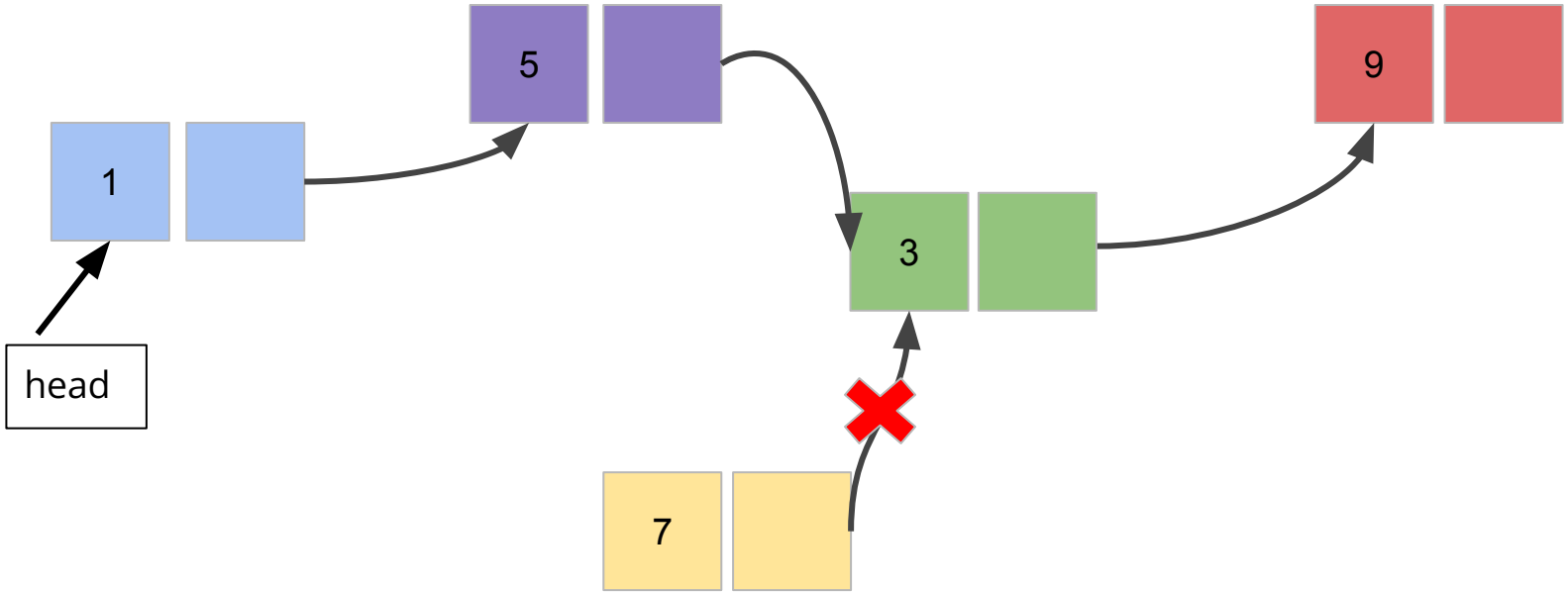
Removal - Linked List



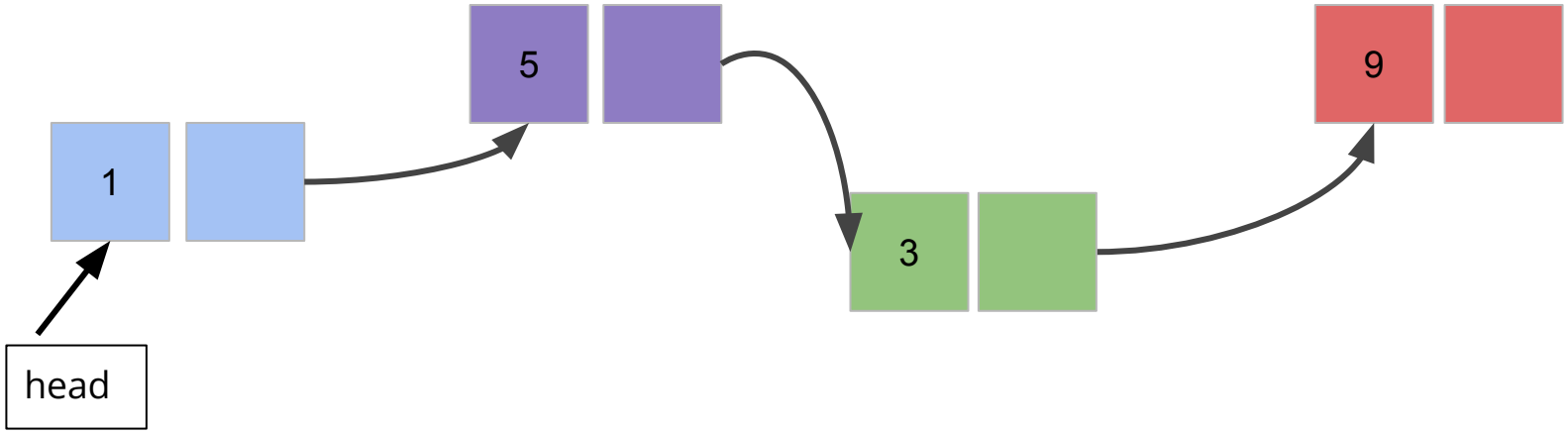
Removal - Linked List



Removal - Linked List



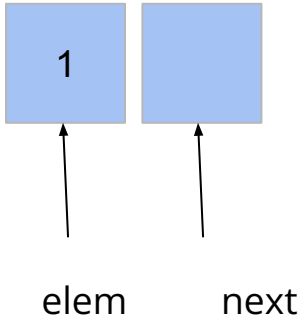
Removal - Linked List



Linked List Creation

```
# Node class design
class Node:
  def __init__(self, e, n):
    self.elem = e
    self.next = n
```

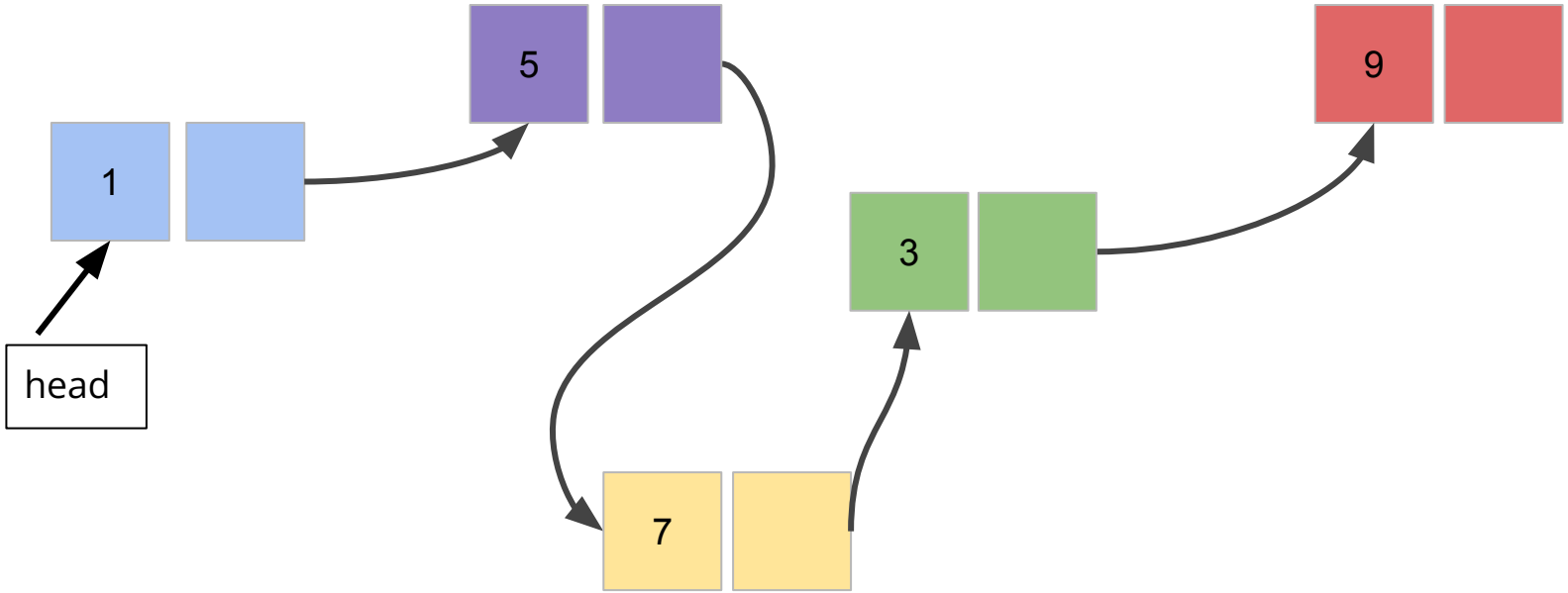
(Address of next node)



Linked List Creation from an Array

```
1 # Creating a list
2 def createList(a):
3     head = Node(a[0], None)
4     tail = head
5     for i in range(1, len(a)):
6         n = Node(a[i], None)
7         tail.next = n
8         tail = tail.next
9     return head
```

Removal - Linked List



Iterating Over a Linked List

```
1 # Iteration over a linked list
2 def iteration(head):
3     temp = head
4     while temp != None:
5         print(temp.elem    )
6         temp = temp.next
```

Counting elements of a Linked List

```
1 # Counting number of element in the list
2 def count(head):
3     count = 0
4     temp = head
5     while temp != None:
6         count += 1
7         temp = temp.next
8     return count
```

Retrieving an element from an index

```
1  # Getting element of an specific index
2  def elemAt(head, idx):
3      count = 0
4      temp = head
5      obj = None
6      while temp != None:
7          if count == idx:
8              obj = temp. elem
9              break
10         temp = temp.next
11         count += 1
12     if obj == None:
13         print("Invalid index")
14     return obj
```

Retrieving a node from an index

```
1 # Getting node of an specific index
2 def nodeAt(head, idx):
3     count = 0
4     temp = head
5     obj = None
6     while temp != None:
7         if count == idx:
8             obj = temp
9             break
10        temp = temp.next
11        count += 1
12    if obj == None:
13        print("Invalid index")
14    return obj
```

Update value of specific index

```
# Setting new element of an specific index
def set(head, idx, elem):
    count = 0
    temp = head
    isUpdated = False
    while temp != None:
        if count == idx:
            temp.elem = elem
            isUpdated = True
            break
        temp = temp.next
        count += 1
    if isUpdated:
        print("Value successfully updated!!!!")
    else:
        print("Invalid index")
```

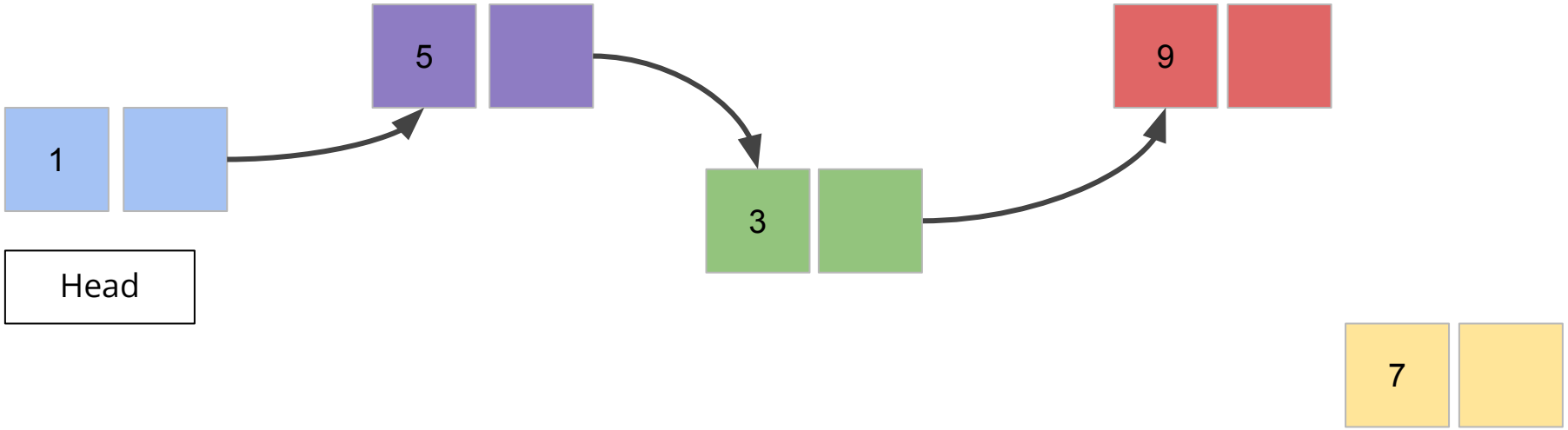
Searching an element in the list

```
1 def contains(head, elem):
2     temp = head
3     while temp != None:
4         if elem == temp.elem:
5             return True
6         temp = temp.next
7     return False
```

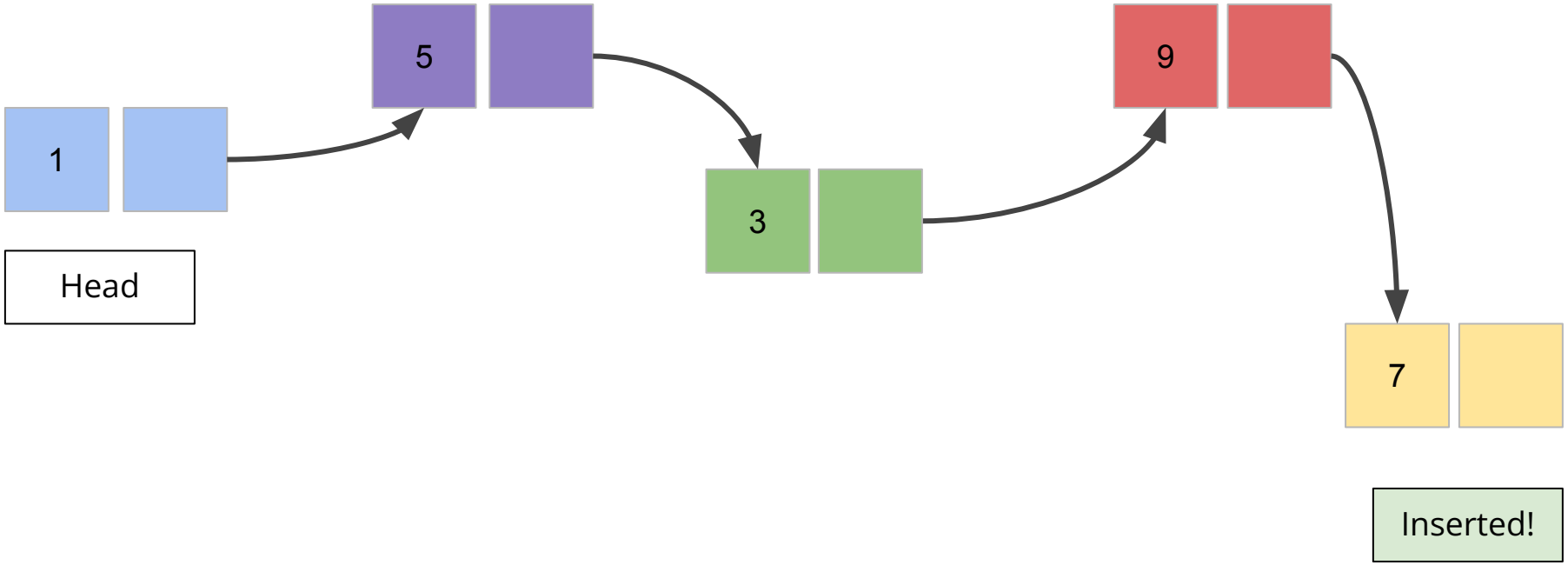
Searching an element in the list

```
# Getting index of an specific element
def indexOf(head, elem):
  temp = head
  count = 0
  while temp != None:
    if elem == temp.elem:
      return count
    count += 1
    temp = temp.next
  return -1 # Here -1 represents the absence of element in the list
```

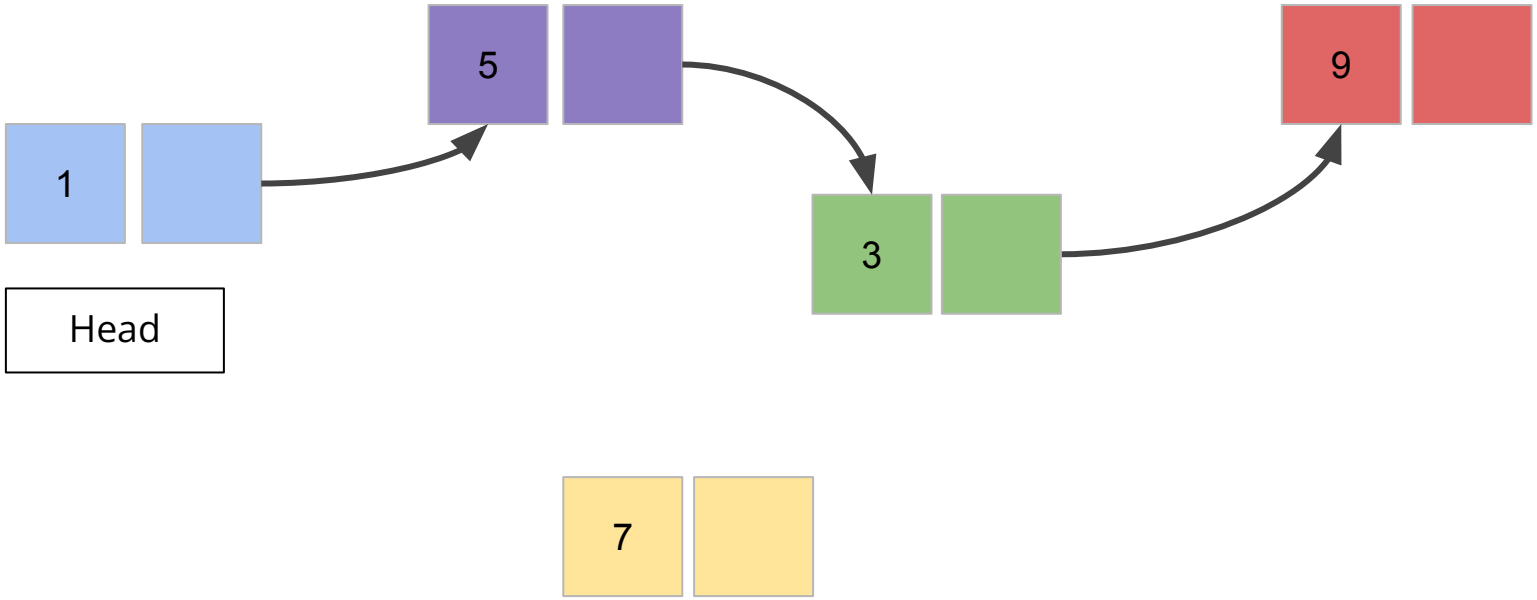
Insertion At the End



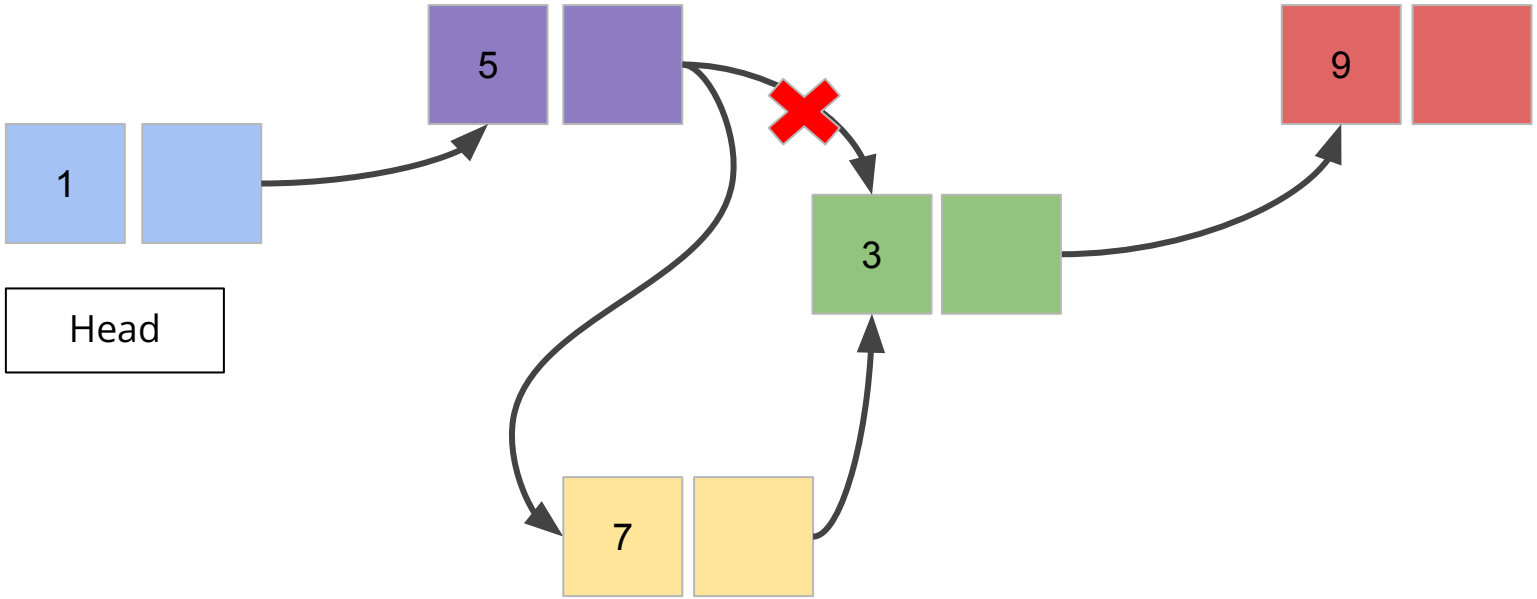
Insertion At the End



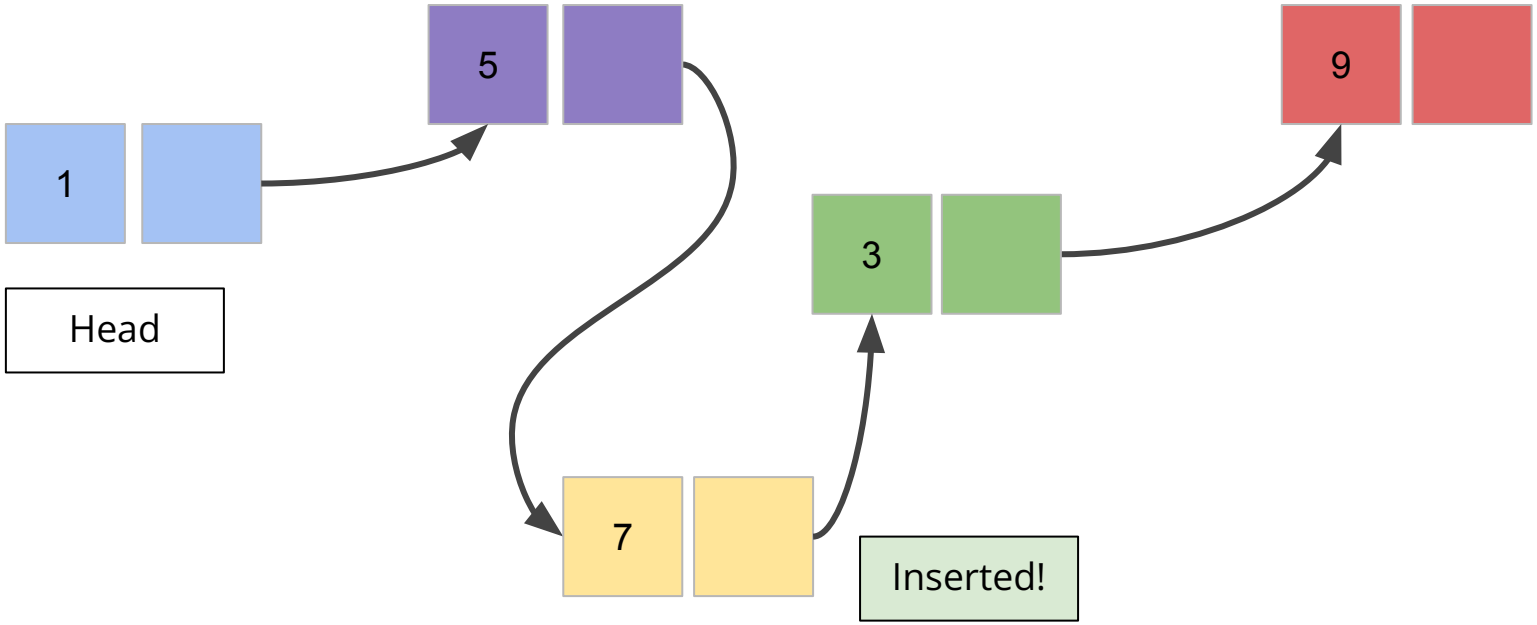
Insertion At the Middle



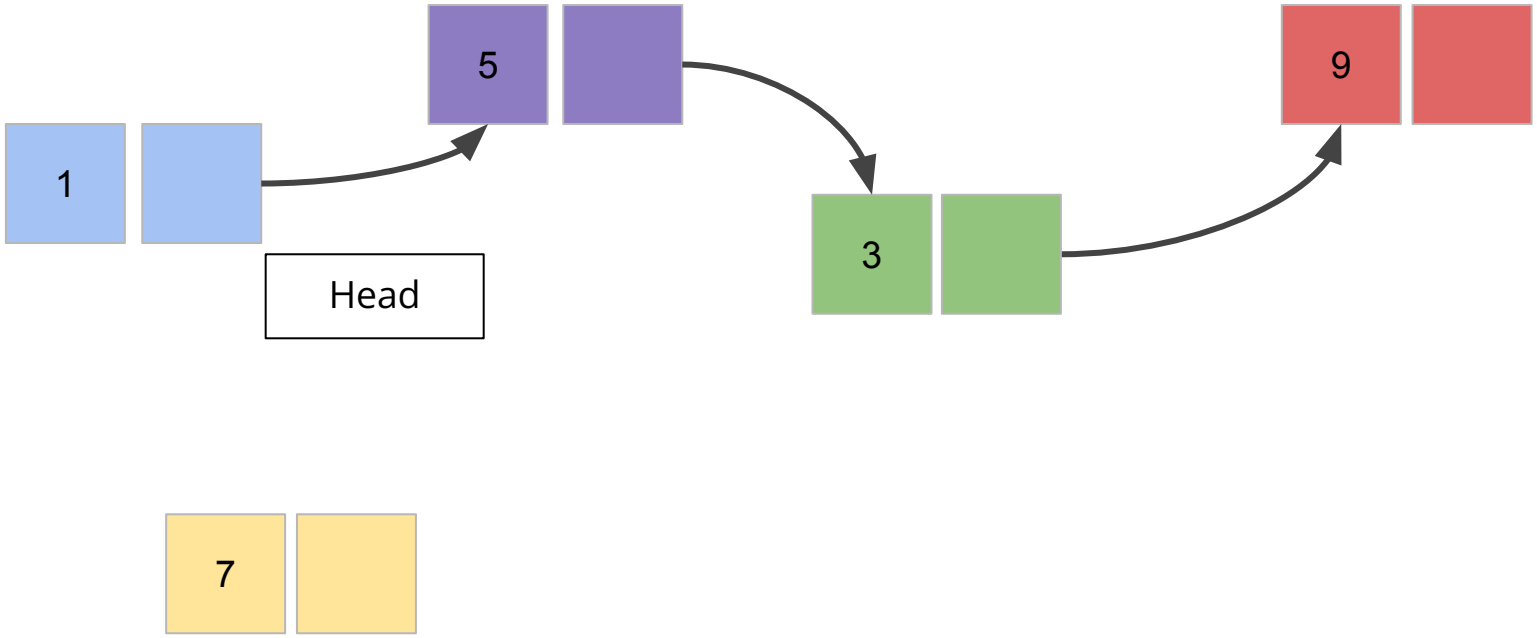
Insertion At the Middle



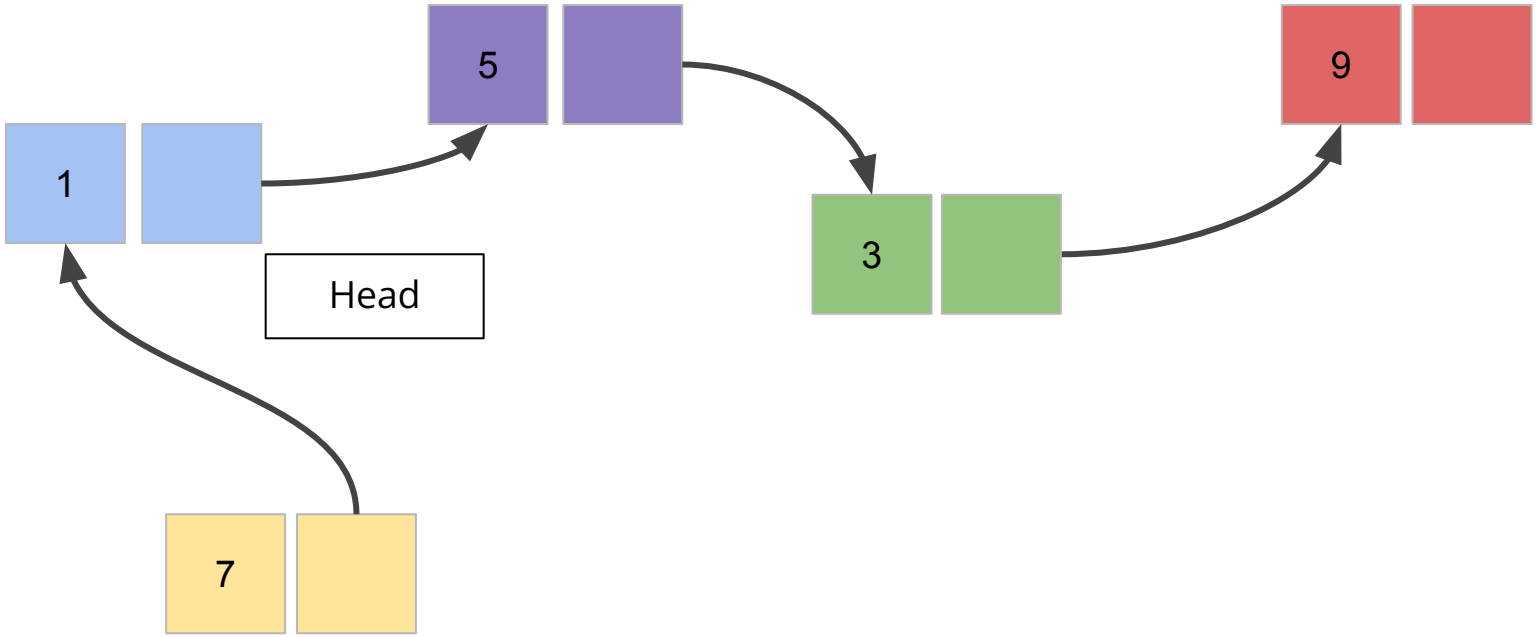
Insertion At the Middle



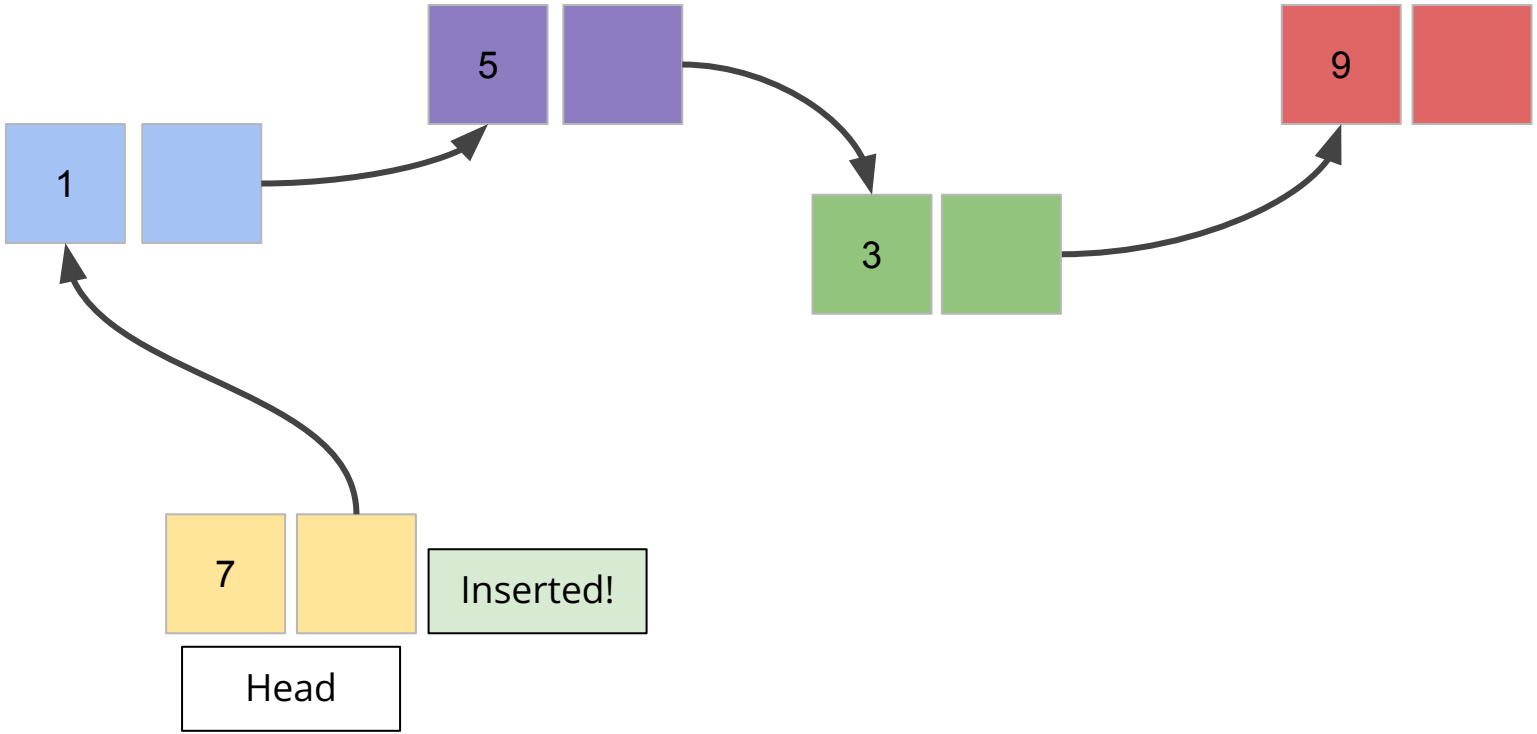
Insertion At the Beginning



Insertion At the Beginning



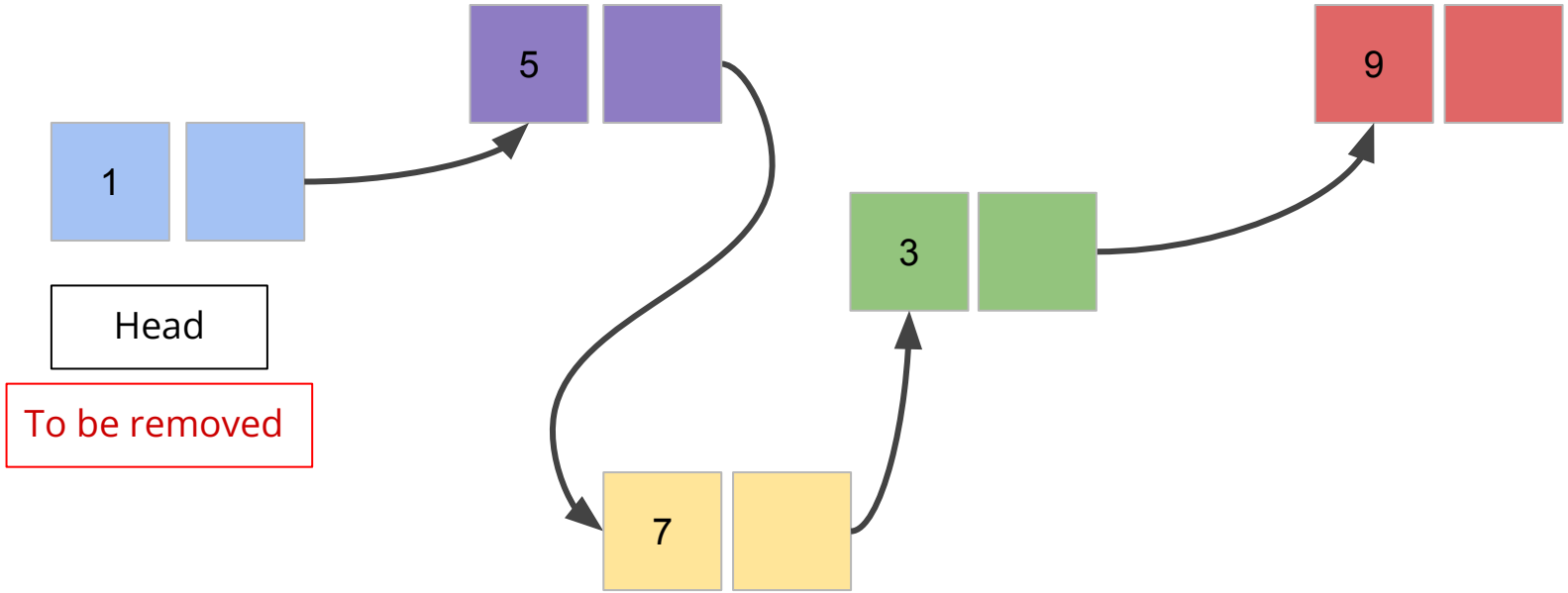
Insertion At the Beginning



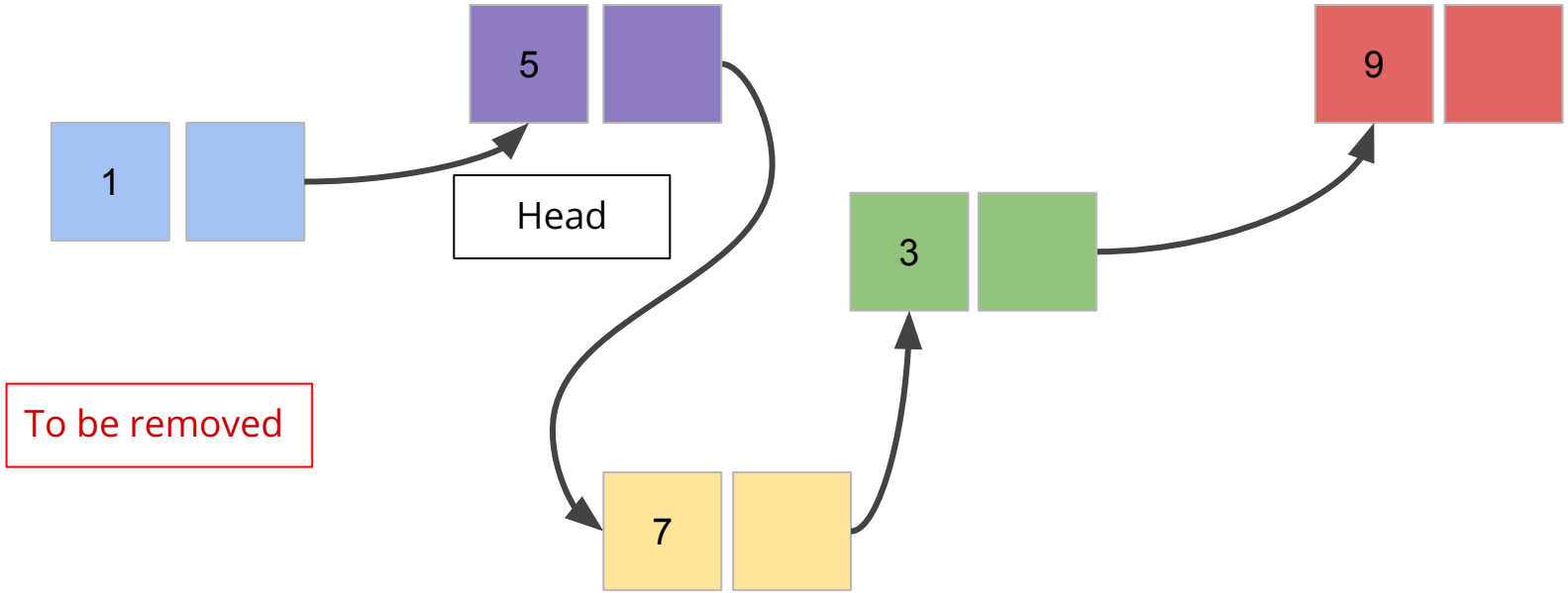
Inserting an element into a list

```
1 def insert(head, elem, idx):
2   total_nodes = count(head)
3   if idx == 0: # Inserting at the beginning
4     n = Node(elem, head)
5     head = n
6   elif idx >= 1 and idx < total_nodes: # Inserting at the middle
7     n = Node(elem, None )
8     n1 = nodeAt(head, idx - 1)
9     n2 = nodeAt(head, idx)
10    n.next = n2
11    n1.next = n
12  elif idx == total_nodes: # Inserting at the end
13    n = Node(elem, None)
14    n1 = nodeAt(head, total_nodes - 1)
15    n1.next = n
16  else:
17    print("Invalid Index")
18  return head
```

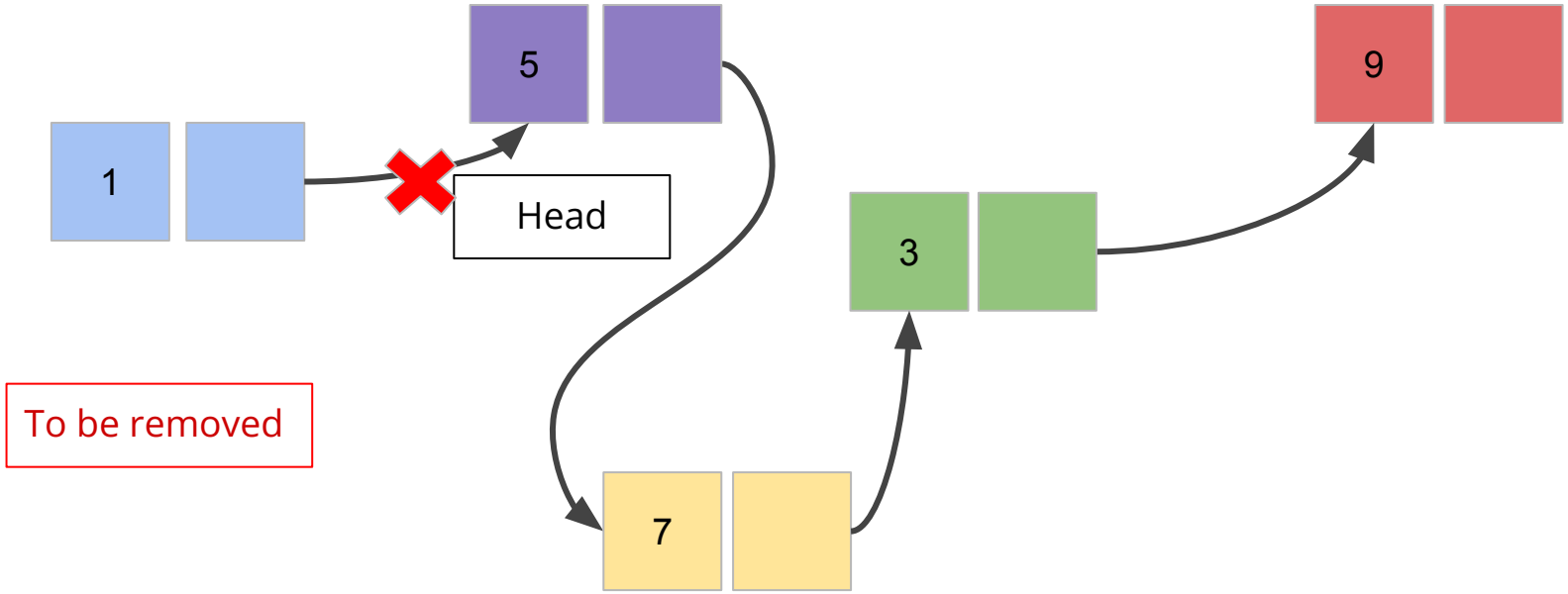
Removing First Element



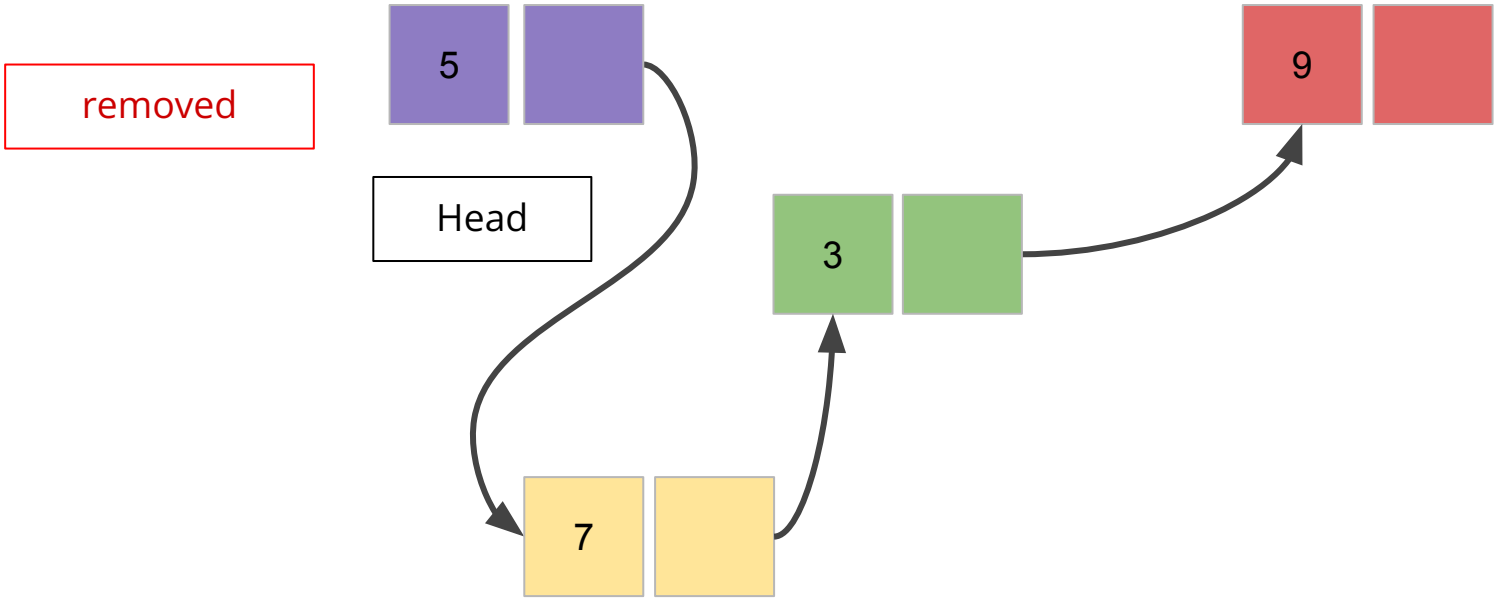
Removing The First Element



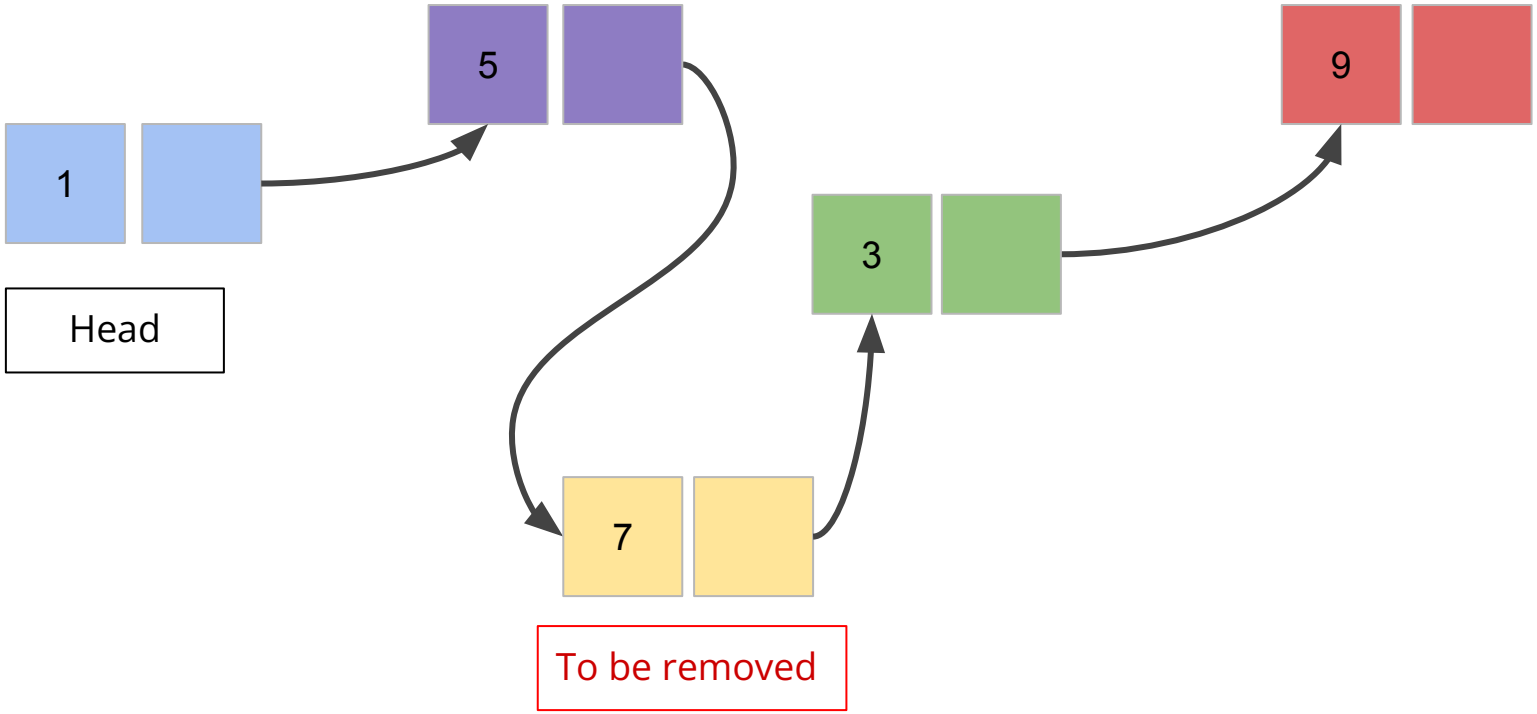
Removing The First Element



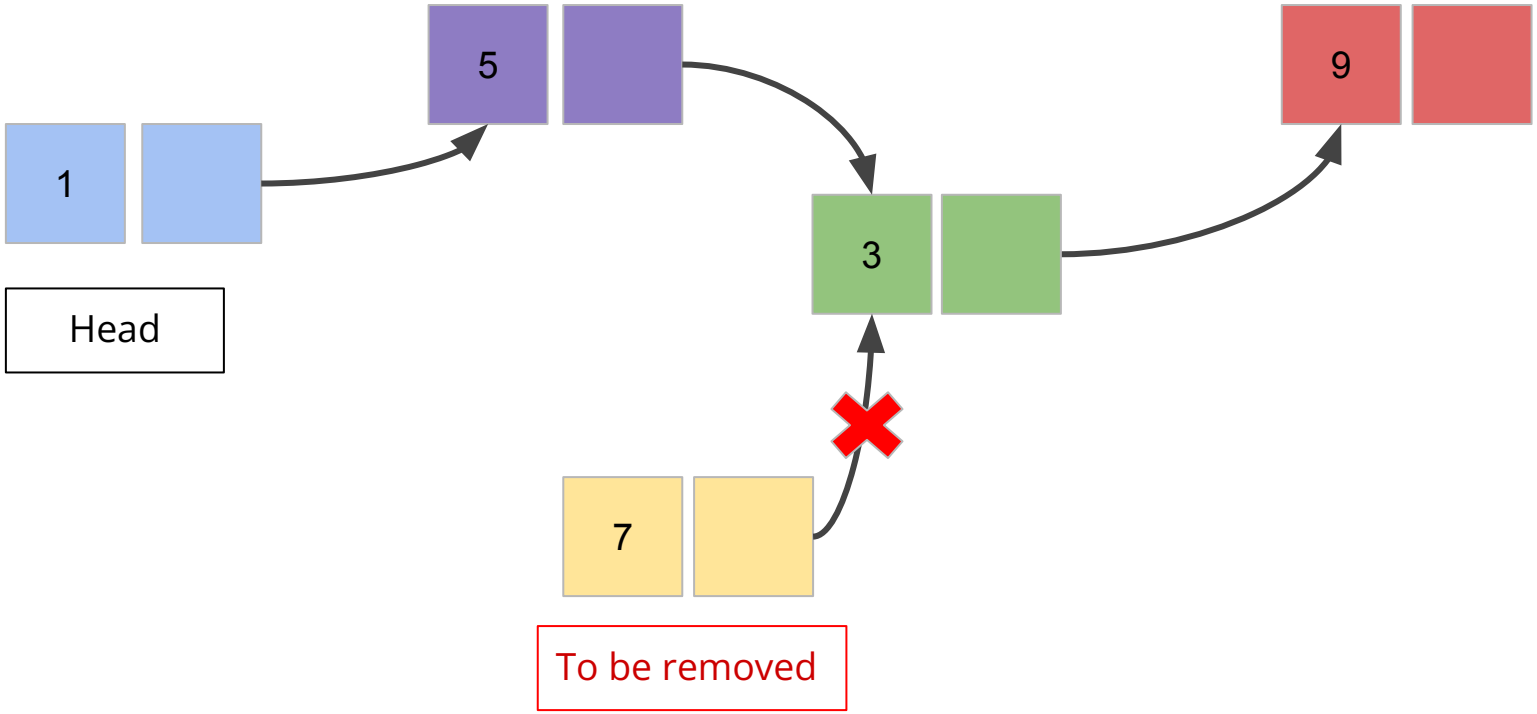
Removing The First Element



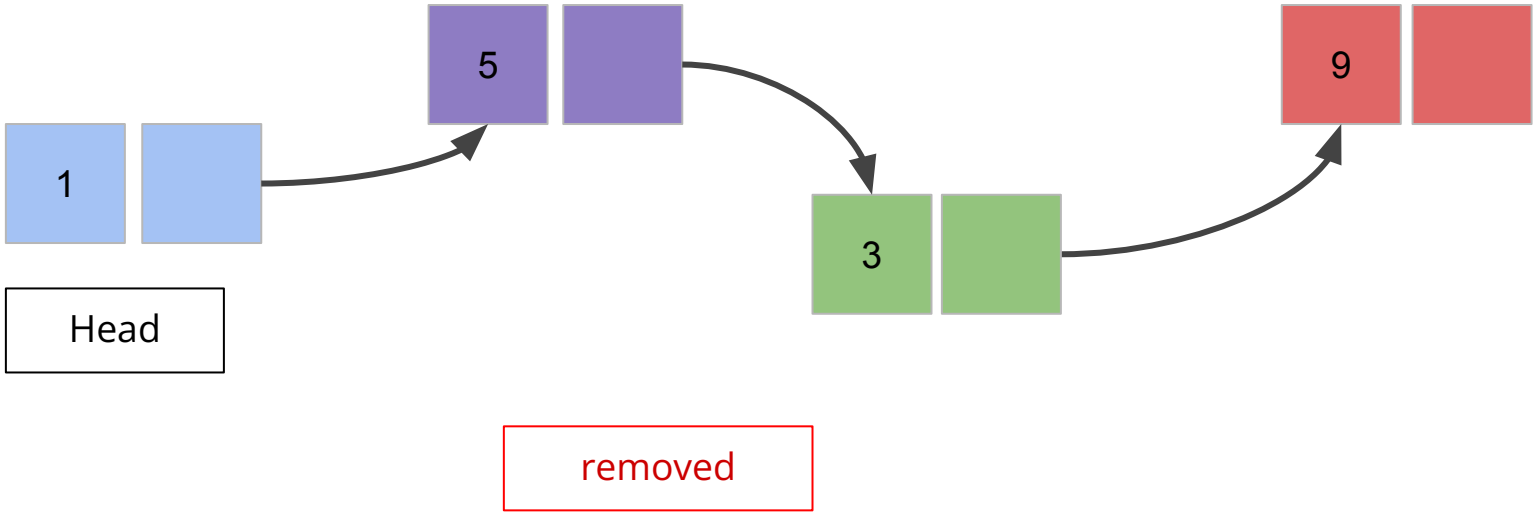
Removing A Middle Element



Removing A Middle Element



Removing A Middle Element



Removing an element from a list

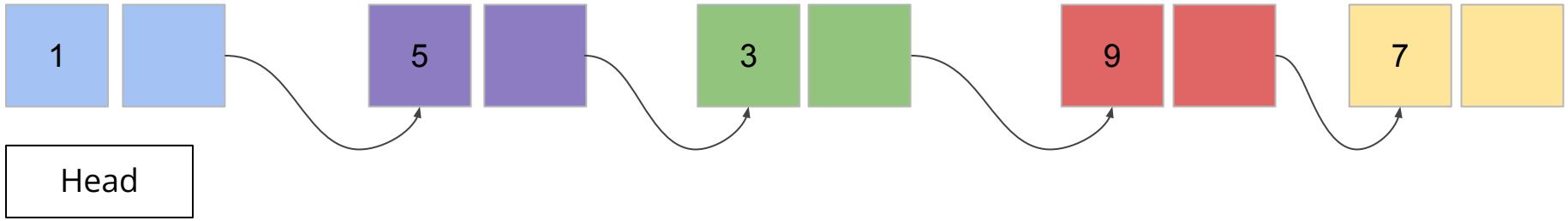
```
1 def remove(head, idx):
2   if idx == 0: # Removing first element
3     head = head.next
4   elif idx >= 1 and idx < count(head): # Removing middle or last element
5     n1 = nodeAt(head, idx - 1)
6     removed_node = n1.next
7     n1.next = removed_node.next
8   else:
9     print("Invalid Index")
10  return head
```

Copying a list

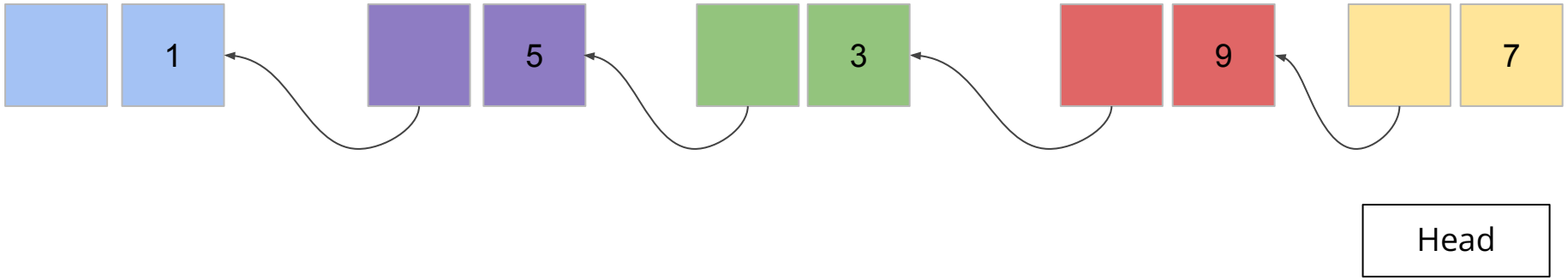
```
1  def copyList(source):
2      copy_head = None
3      copy_tail = None
4      temp = source
5      while temp != None:
6          n = Node(temp.elem, None)
7          if copy_head == None:
8              copy_head = n
9              copy_tail = copy_head
10         else:
11             copy_tail.next = n
12             copy_tail = copy_tail.next
13         temp = temp.next
14     return copy_head
```

Reversing a list

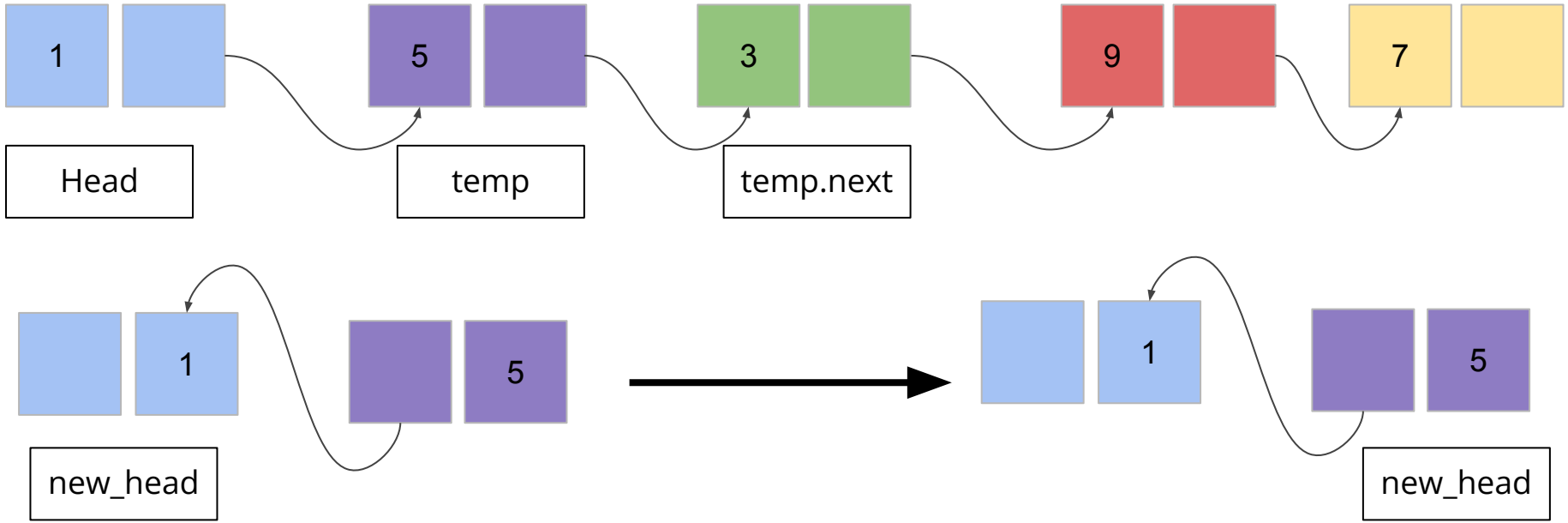
Before



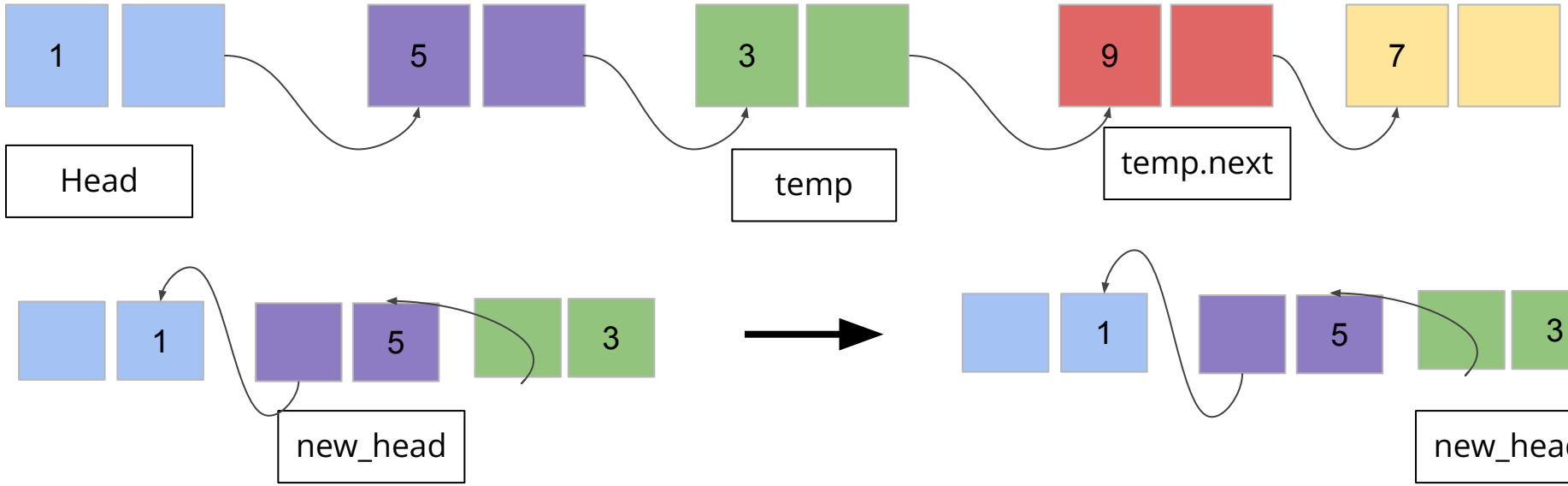
After



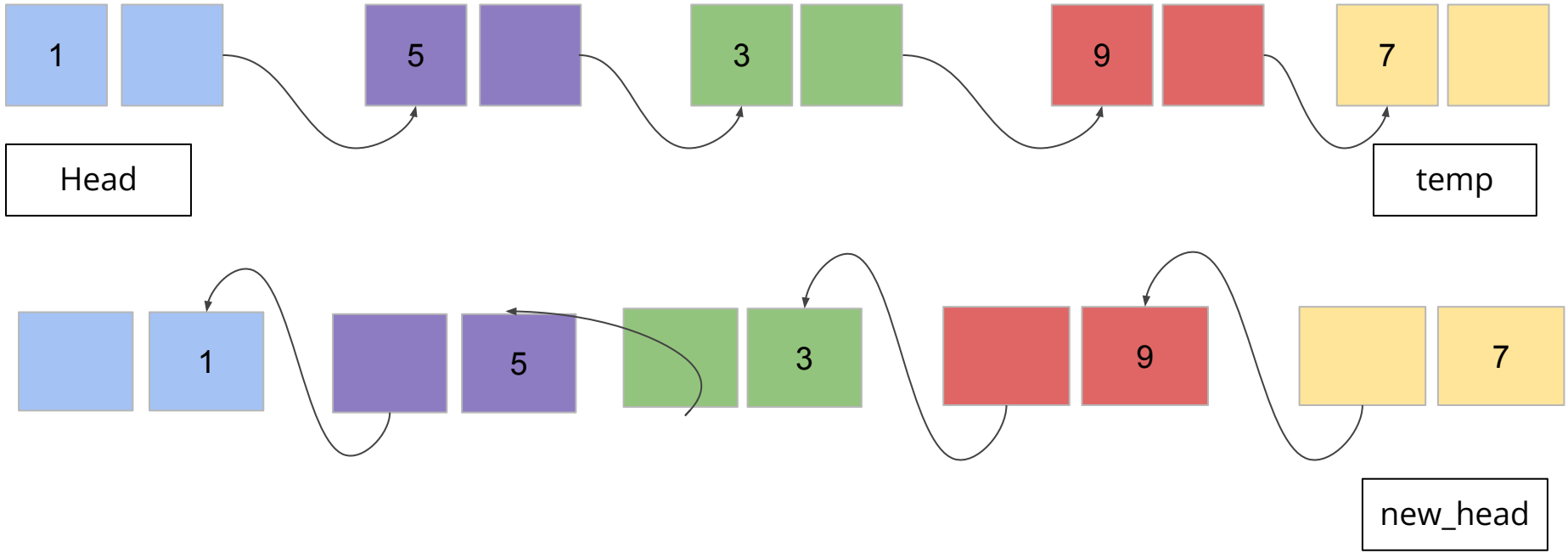
Reversing a list - out of place



Reversing a list - out of place



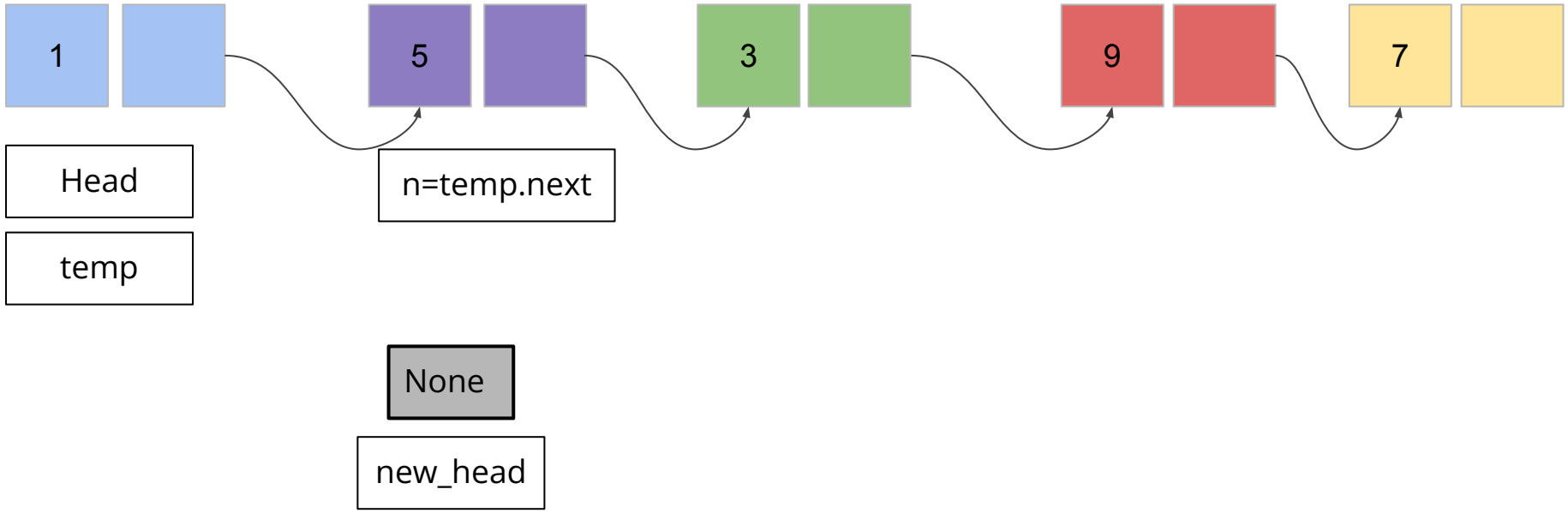
Reversing a list - out of place



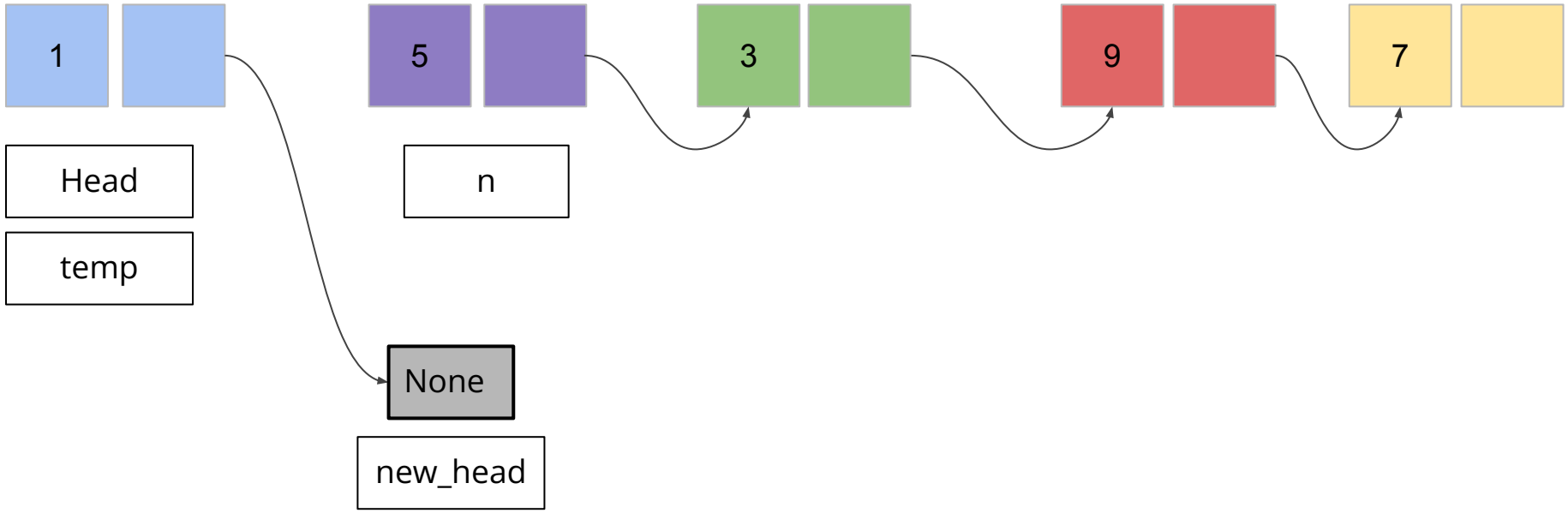
Reversing a list - out of place

```
1 def reverse_out_of_place(head):
2     new_head = Node(head.elem, None)
3     temp = head.next
4     while temp != None:
5         n = Node(temp.elem, new_head)
6         new_head = n
7         temp = temp.next
8     return new_head
```

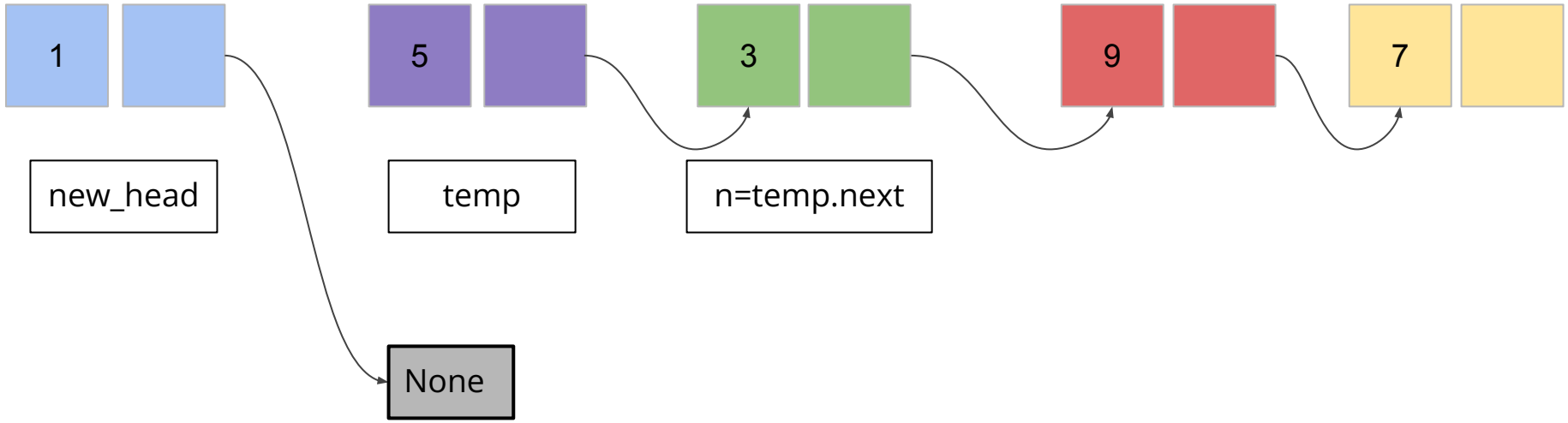
Reversing a list - in place



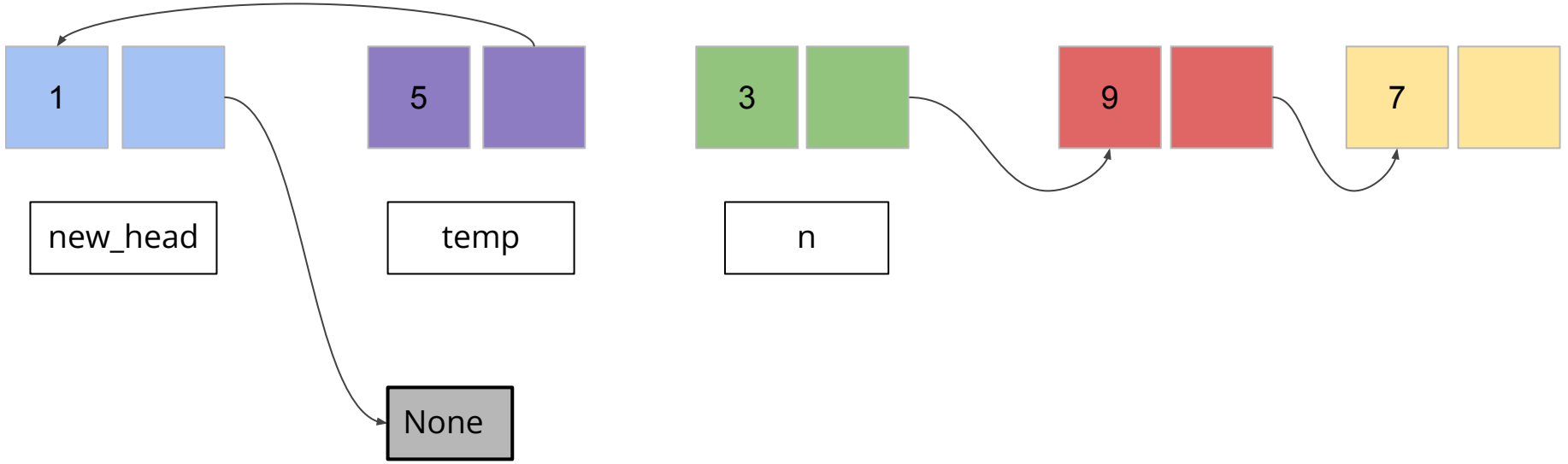
Reversing a list - in place



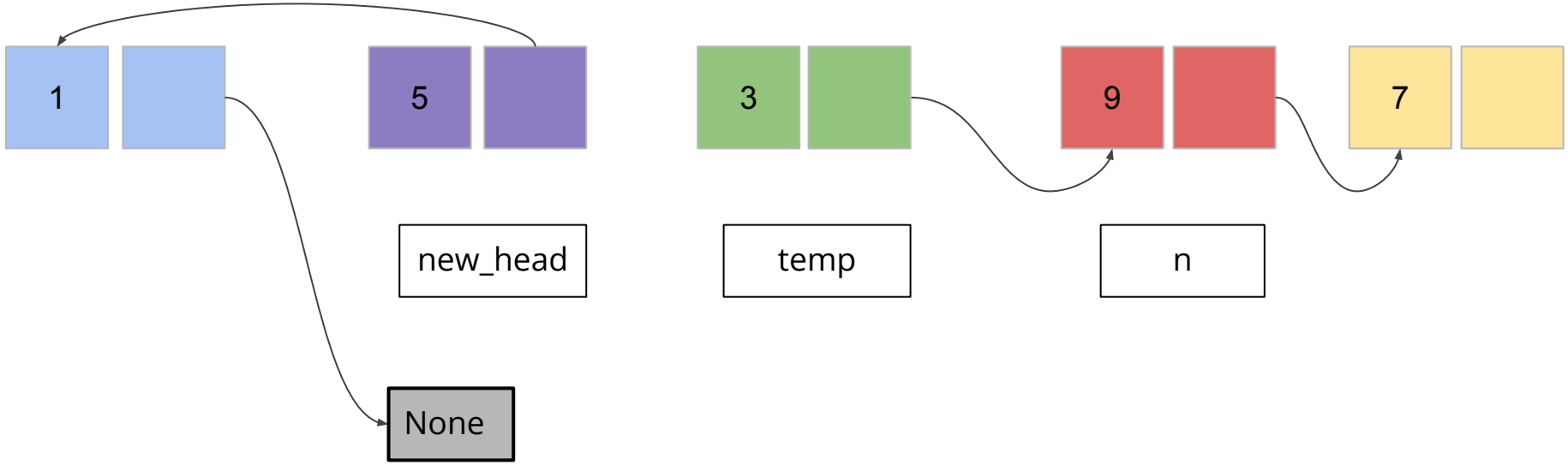
Reversing a list - in place



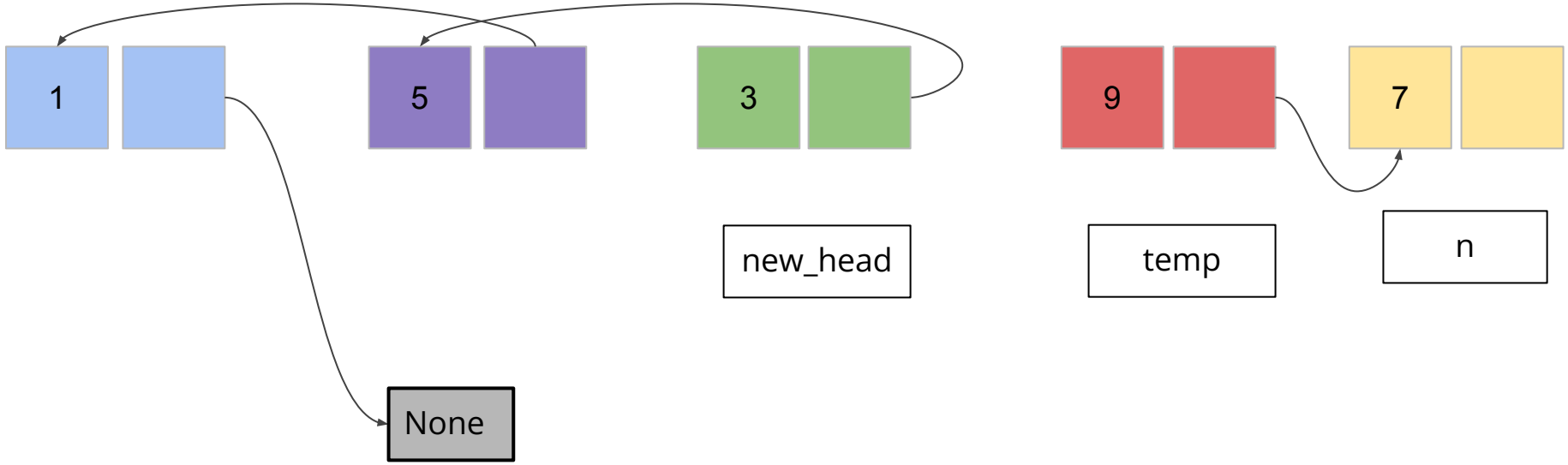
Reversing a list - in place



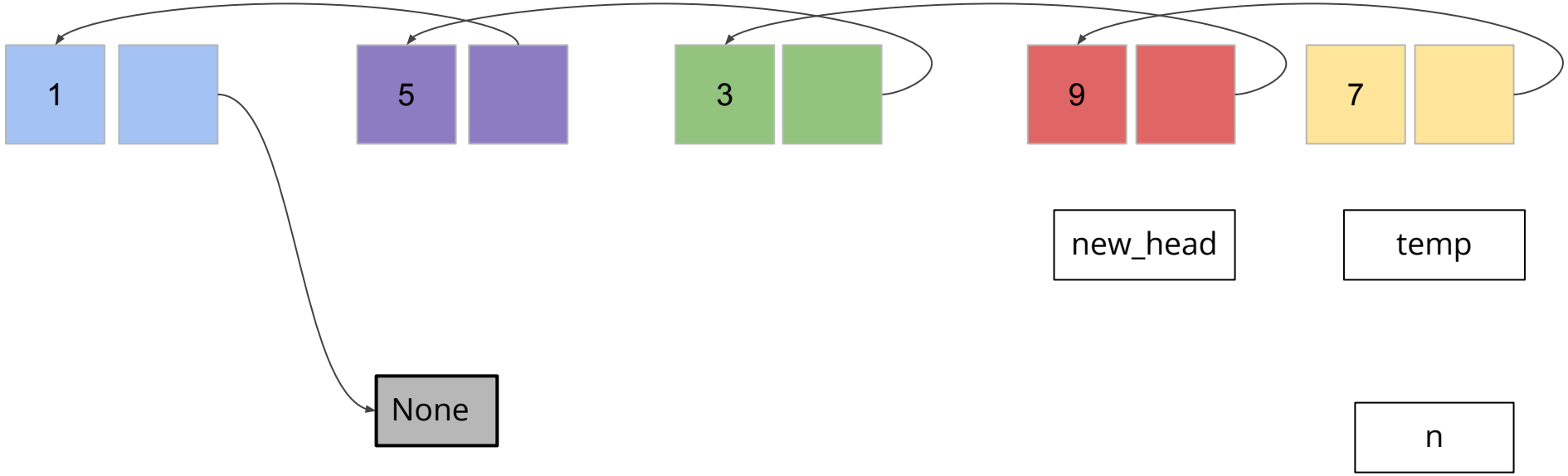
Reversing a list - in place



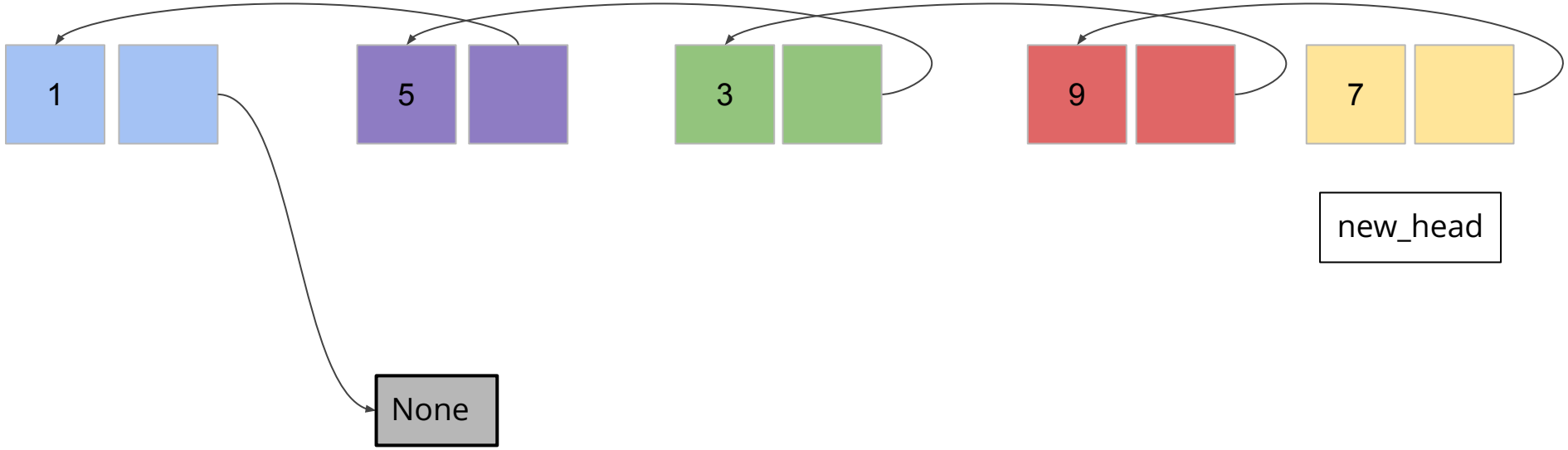
Reversing a list - in place



Reversing a list - in place



Reversing a list - in place

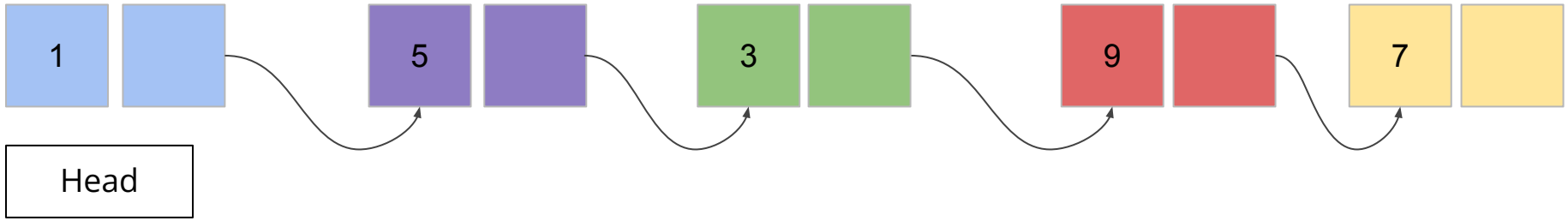


Reversing a list - in place

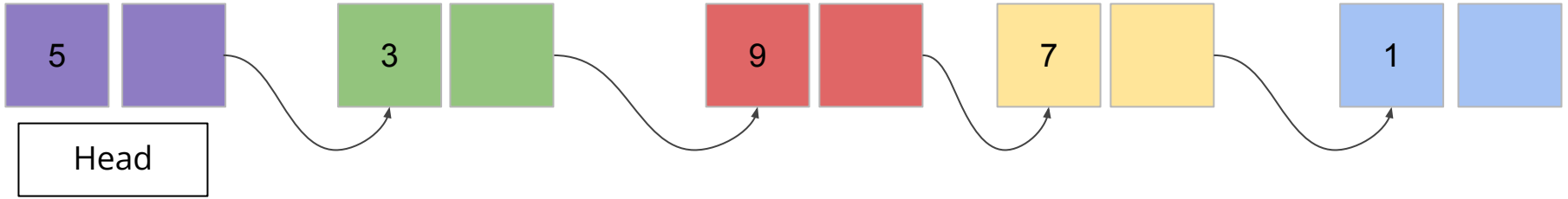
```
1 def reverse_in_place(head):
2     new_head = None
3     temp = head
4     while temp != None:
5         n = temp.next
6         temp.next = new_head
7         new_head = temp
8         temp = n
9     return new_head
```

Rotating a list - left

Before



After

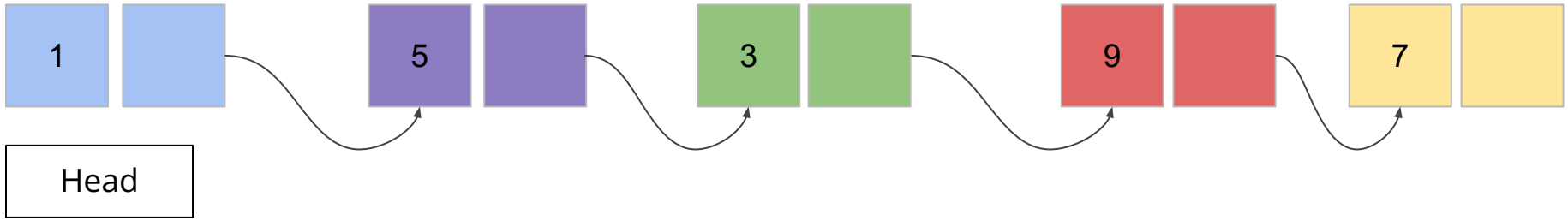


Rotating a list - left

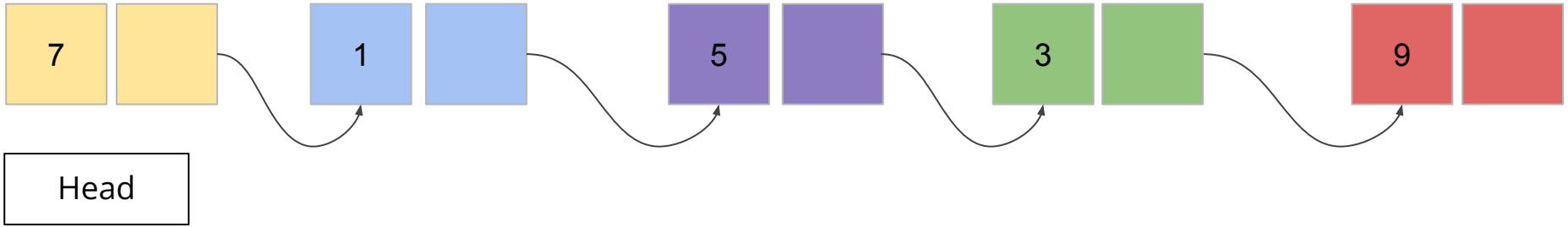
```
1 def rotate_left(head):
2     new_head = head.next
3     temp = new_head
4     while temp.next != None:
5         temp = temp.next
6     temp.next = head
7     head.next = None
8     head = new_head
9     return head
```

Rotating a list - right

Before



After



Rotating a list - right

```
1 def rotate_right(head):
2   last_node = head.next
3   second_last_node = head
4   while last_node.next != None:
5     last_node = last_node.next
6     second_last_node = second_last_node.next
7   last_node.next = head
8   second_last_node.next = None
9   head = last_node
10  return head
```